Microservicios con Spring

© JMA 2020. All rights reserved

http://spring.io

SPRING CON SPRING BOOT

© JMA 2020. All rights reserved

Spring

- Spring Framework es un marco de desarrollo de aplicaciones Java que facilita la creación de aplicaciones empresariales robustas y modulares al proporcionar un conjunto integral de funcionalidades, como inversión de control, contenedor de beans o gestión de transacciones. Utilizar Spring y Spring Boot proporciona ventajas como la flexibilidad, la modularidad, la escalabilidad y una amplia gama de características integradas que aceleran el desarrollo de aplicaciones empresariales.
- Inicialmente era un ejemplo hecho para el libro "J2EE design and development" de Rod Johnson en 2003, que defendía alternativas a la "visión oficial" de aplicación JavaEE basada en EJBs. Actualmente es un framework open source que facilita el desarrollo de aplicaciones java JEE & JSE (no esta limitado a aplicaciones Web ni a java).
- Provee de un contenedor encargado de manejar el ciclo de vida de los objetos (beans) para que los desarrolladores se enfoquen a la lógica de negocio. Permite integración con diferentes frameworks. Surge como una alternativa a EJB's
- Actualmente es un framework completo compuesto por múltiples módulos/proyectos que cubre todas las capas de la aplicación, con decenas de desarrolladores y miles de descargas al día.

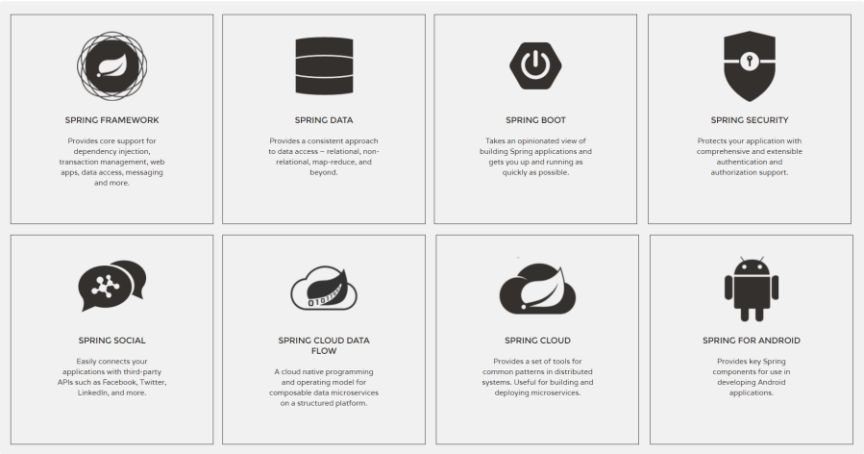
© JMA 2020. All rights reserved

Características

- Ligero
  - No se refiere a la cantidad de clases sino al mínimo impacto que se tiene al integrar Spring.
- No intrusivo
  - Generalmente los objetos que se programan no tienen dependencias de clases específicas de Spring
- Flexible
  - Aunque Spring provee funcionalidad para manejar las diferentes capas de la aplicación (vista, lógica de negocio, acceso a datos) no es necesario usarlo para todo. Brinda la posibilidad de utilizarlo en la capa o capas que queramos.
- Multiplataforma
  - Escrito en Java, corre sobre JVM

© JMA 2020. All rights reserved

Proyectos



© JMA 2020. All rights reserved

Módulos necesarios

- Spring Framework
  - Spring Core
    - Contenedor IoC (inversión de control) - inyector de dependencia
  - Spring MVC
    - Framework basado en MVC para aplicaciones web y servicios REST
- Spring Data
  - Simplifica el acceso a los datos: JPA, bases de datos relacionales / NoSQL, nube
- Spring Boot
  - Simplifica el desarrollo de Spring: inicio rápido con menos codificación

© JMA 2020. All rights reserved

Spring Boot

- Spring tiene una gran cantidad de módulos que implican multitud de configuraciones. Estas configuraciones pueden requerir mucho tiempo, pueden ser desconocidas para principiantes y suelen ser repetitivas.
- La solución de Spring es Spring Boot, que aplica el concepto de Convention over Configuration (CoC).
- CoC es un paradigma de programación que minimiza las decisiones que tienen que tomar los desarrolladores, simplificando tareas.
- No obstante, la flexibilidad no se pierde, ya que a pesar de otorgar valores por defecto, siempre se puede configurar de forma extendida.
- De esta forma se evita la repetición de tareas básicas a la hora de construir un proyecto.
- Spring Boot es una herramienta que nace con la finalidad de simplificar aun más el desarrollo de aplicaciones basadas en el framework Spring Core: que el desarrollador solo se centre en el desarrollo de la solución, olvidándose por completo de la compleja configuración que actualmente tiene Spring Core para poder funcionar.

© JMA 2020. All rights reserved

Spring Boot

- Resolución de dependencias:
  - Con Spring Boot solo hay que determinar que tipo de proyecto estaremos utilizando y el se encarga de resolver todas las librerías/dependencias para que la aplicación funcione.
- Configuración:
  - Spring Boot cuenta con un complejo módulo que autoconfigura todos los aspectos de nuestra aplicación para poder simplemente ejecutar la aplicación, sin tener que definir absolutamente nada.
- Despliegue:
  - Spring Boot se puede ejecutar como una aplicación Stand-alone, pero también es posible ejecutar aplicaciones web, ya que es posible desplegar las aplicaciones mediante un servidor web integrado, como es el caso de Tomcat, Jetty o Undertow.

© JMA 2020. All rights reserved

Spring Boot

- Métricas:
  - Por defecto, Spring Boot cuenta con servicios que permite consultar el estado de salud de la aplicación, permitiendo saber si la aplicación está encendida o apagada, memoria utilizada y disponible, número y detalle de los Bean's creado por la aplicación, controles para el prendido y apagado, etc.
- Extensible:
  - Spring Boot permite la creación de complementos, los cuales ayudan a que la comunidad de Software Libre cree nuevos módulos que faciliten aún más el desarrollo.
- Productividad:
  - Herramientas de productividad para desarrolladores como Spring Initializr, Lombok, LiveReload y Auto Restart, funcionan en su IDE favorito: Spring Tool Suite, IntelliJ IDEA y NetBeans.

© JMA 2020. All rights reserved

Dependencias: starters

- Los starters son un conjunto de descriptores de dependencias convenientes (versiones compatibles, ya probadas) que se pueden incluir en la aplicación.
- Se obtiene una ventanilla única para el módulo de Spring y la tecnología relacionada que se necesita, sin tener que buscar a través de códigos de ejemplo y copiar/pegar cargas de descriptores de dependencia.
- Por ejemplo, si desea comenzar a utilizar Spring con JPA para un acceso CRUD a base de datos, basta con incluir la dependencia spring-boot-starter-data-jpa en el proyecto.

© JMA 2020. All rights reserved

devtools

- Al realizar nuevos cambios en nuestra aplicación, podemos hacer que el arranque se reinicie automáticamente. Para eso es necesario incluir una dependencia Maven extra: spring-boot-devtools.
- Durante el tiempo de ejecución, Spring Boot supervisa la carpeta que se encuentra en classpath (en maven, las carpetas que están en la carpeta "target"). Solo necesitamos activar la compilación de las fuentes en los cambios que causarán la actualización de la carpeta 'destino' y Spring Boot reiniciará automáticamente la aplicación. Si estamos utilizando Eclipse IDE, la acción de guardar puede desencadenar la compilación.
- El módulo spring-boot-devtools incluye un servidor LiveReload incorporado que activa una actualización del navegador cuando se cambia un recurso. Las extensiones del navegador LiveReload están disponibles gratuitamente para Chrome, Firefox y Safari desde:
  http://livereload.com/extensions/
- Para configurar LiveReload:
  spring.devtools.restart.additional-paths=META-INF/resources/**
  spring.devtools.livereload.enabled=true # por defecto

© JMA 2020. All rights reserved

Spring Tools

https://spring.io/tools

- Spring Tools 4 es la próxima generación de herramientas Spring para los entorno IDE de codificación favoritos. Proporciona soporte de primera clase para el desarrollo de aplicaciones empresariales basadas en Spring, ya sea que se prefiera Eclipse, Visual Studio Code o Theia IDE.
  - Help → Eclipse Marketplace …
    - Spring Tools 4 for Spring Boot
- Spring Tool Suite (STS) es un IDE basado en la versión Java EE de Eclipse, pero altamente personalizable para trabajar con Spring Framework.
  - IDE gratuito, personalización del Eclipse

© JMA 2020. All rights reserved

Crear proyecto

- Desde web:
  - https://start.spring.io/
  - Descomprimir en el workspace
  - Import → Maven → Existing Maven Project
- Desde Eclipse:
  - New Project → Sprint Boot → Spring Started Project
- Dependencias
  - Web
  - JPA
  - JDBC (o proyecto personalizado)

© JMA 2020. All rights reserved

Application

```
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication implements CommandLineRunner {

	public static void main(String[] args) {
		SpringApplication.run(ApiHrApplication.class, args);
	}

	@Override
	public void run(String... args) throws Exception {
		// Opcional: Procesar los args una vez arrancado SprintBoot

	}

}
```

© JMA 2020. All rights reserved

Configuración

- @Configuration: Indica que esta es una clase usada para configurar el contenedor Spring.
- @ComponentScan: Escanea los paquetes de nuestro proyecto en busca de los componentes que hayamos creado, ellos son, las clases que utilizan las siguientes anotaciones: @Component, @Service, @Controller, @Repository.
- @EnableAutoConfiguration: Habilita la configuración automática, esta herramienta analiza el classpath y el archivo application.properties para configurar nuestra aplicación en base a las librerías y valores de configuración encontrados, por ejemplo: al encontrar el motor de bases de datos H2 la aplicación se configura para utilizar este motor de datos, al encontrar Thymeleaf se crearan los beans necesarios para utilizar este motor de plantillas para generar las vistas de nuestra aplicación web.
- @SpringBootApplication: Es el equivalente a utilizar las anotaciones: @Configuration, @EnableAutoConfiguration y @ComponentScan

© JMA 2020. All rights reserved

Configuración

- Editar src/main/resources/application.properties:

```
server.port=${PORT:8080}
# Oracle settings
spring.datasource.url=jdbc:oracle:thin:@localhost:1521:xe
spring.datasource.username=hr
spring.datasource.password=hr
spring.datasource.driver-class=oracle.jdbc.driver.OracleDriver

# MySQL settigs
spring.datasource.url=jdbc:mysql://localhost:3306/sakila
spring.datasource.username=root
spring.datasource.password=root
#spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

logging.pattern.console=%d{yyyy-MM-dd HH:mm:ss} %-5level %logger{36} - %msg%n
logging.level.org.hibernate.SQL=debug
```

- Repetir con src/test/resources/application.properties
- Eclipse: Run Configurations → Arguments → VM Arguments: -DPORT=8888

© JMA 2020. All rights reserved

Instalación de MySQL

- Descargar e instalar:
  - https://mariadb.org/download/
- Incluir en la seccion [mysqld] de %MYSQL_ROOT%/data/my.ini
  - default_time_zone='+01:00'
- Descargar bases de datos de ejemplos:
  - https://dev.mysql.com/doc/index-other.html
- Instalar bases de datos de ejemplos:
  - mysql -u root -p < employees.sql
  - mysql -u root -p < sakila-schema.sql
  - mysql -u root -p < sakila-data.sql

© JMA 2020. All rights reserved

Instalación con Docker

- Docker Toolbox
  - Windows 10 Pro ++: https://docs.docker.com/docker-for-windows/install/
  - Otras: https://github.com/docker/toolbox/releases
- Ejecutar Docker QuickStart
- Para crear el contenedor de MySQL con la base de datos Sakila:
  - docker run -d --name mysql-sakila -e MYSQL_ROOT_PASSWORD=root -p 3306:3306 jamarton/mysql-sakila
- Para crear el contenedor de MongoDB:
  - docker run -d --name mongodb -p 27017:27017 mongo
- Para crear el contenedor de Redis:
  - docker run --name redis -p 6379:6379 -d redis
  - docker run --name redis-insight -d -v redisinsight:/db -p 6380:8001 redislabs/redisinsight:latest
  - docker run -d --name redis-commander -p 8081:8081 rediscommander/redis-commander:latest
- Para crear el contenedor de RabbitMQ:
  - docker run -d --hostname rabbitmq --name rabbitmq -p 4369:4369 -p 5671:5671 -p 5672:5672 -p 15671:15671 -p 15672:15672 -p 25672:25672 -e RABBITMQ_DEFAULT_USER=admin -e RABBITMQ_DEFAULT_PASS=curso rabbitmq:management-alpine

© JMA 2020. All rights reserved

ESTILOS DE COMUNICACIÓN

© JMA 2020. All rights reserved

Estilos de comunicación

- El cliente y los servicios, o los servicios entre si, pueden comunicarse a través de muchos tipos diferentes de comunicación, cada uno destinado a un escenario y unos objetivos distintos. Inicialmente, estos tipos de comunicaciones se pueden clasificar por diferentes criterios.
- El primer criterio define si el proceso es sincrónico o asincrónico:
  - Proceso síncrono: HTTP es el protocolo síncrono mas utilizado. El cliente envía una solicitud y espera una respuesta del servicio, solo puede continuar su tarea cuando recibe la respuesta del servidor. Es independiente de la ejecución de código de cliente, que puede ser síncrono (el subproceso está bloqueado) o asíncrono (el subproceso no está bloqueado y la respuesta dispara una devolución de llamada).
  - Proceso asíncrono: Otros protocolos como AMQP (un protocolo compatible con muchos sistemas operativos y entornos de nube) usan mensajes asíncronos. Normalmente el código de cliente o el remitente del mensaje no espera ninguna respuesta. Simplemente se envía el mensaje a una cola de un agente de mensajes, que son escuchadas por los consumidores.
- El segundo criterio define si la comunicación tiene un único receptor o varios:
  - Receptor único 1:1 (comando, punto a punto, P2P): Cada solicitud debe ser procesada por un receptor o servicio exactamente (como en el patrón Command).
  - Varios receptores 1:N (eventos, publicación/subscripción, Pub/Sub): Cada solicitud puede ser procesada por entre cero y varios receptores. Este tipo de comunicación debe ser asíncrona (basada en un bus de eventos o un agente de mensajes).

© JMA 2020. All rights reserved

Criterio según su base

- Basados en Recursos: Los servicios exponen información, documentos que incluyen tanto identificadores de datos como de acciones (enlaces y formularios), las operaciones CRUD están predefinidas. REST es un estilo arquitectónico que separa las preocupaciones del consumidor y del proveedor de la API al depender de comandos que están integrados en el protocolo de red subyacente. REST (Representational State Transfer) es extremadamente flexible en el formato de sus cargas útiles de datos, lo que permite una variedad de formatos de datos populares como JSON y XML, entre otros.
- Basados en Procedimientos: Las llamadas a procedimiento remoto, o RPC, generalmente requieren que los desarrolladores ejecuten bloques específicos de código en otro sistema: operaciones. RPC es independiente del protocolo, lo que significa que tiene el potencial de ser compatible con muchos protocolos, pero también pierde los beneficios de usar capacidades de protocolo nativo (por ejemplo, almacenamiento en caché). La utilización de diferentes estándares da como resultado un acoplamiento más estrecho entre los consumidores y los proveedores de API y las tecnologías implicadas, lo que a su vez sobrecarga a los desarrolladores involucrados en todos los aspectos de un ecosistema de APIs impulsado por RPC. Los patrones de arquitectura de RPC se pueden observar en tecnologías API populares como SOAP, GraphQL y gRPC.
- Basados en Eventos/Streaming: a veces denominadas arquitecturas de eventos, en tiempo real, de transmisión, asíncronas o push, las APIs impulsadas por eventos no esperan a que un consumidor de la API las llame antes de entregar una respuesta. En cambio, una respuesta se desencadena por la ocurrencia de un evento. Estos servicios exponen eventos, con una información mínima, a los que los clientes pueden suscribirse para recibir actualizaciones cuando cambian los valores del servicio. Hay un puñado de variaciones para este estilo que incluyen (entre otras) reactivo, publicador/suscriptor, notificación de eventos y CQRS.

© JMA 2020. All rights reserved

Evolución histórica de los servicios

- Precursores:
  - RPC: Llamadas a Procedimientos Remotos
  - Binarios: CORBA, Java RMI, .NET Remoting
  - XML-RPC: Precursor del SOAP
- Actuales:
  - Servicios Web XML o Servicios SOAP
  - Servicios Web REST o API REST
    - WebHooks
  - Servicios GraphQL
  - Servicios gRPC

AÑO	Descripción
1976	Aparición de RPC (Remote Procedure Call) en Sistema Unix
1990	Aparición de DCE (Distributed Computing Environment) que es un sistema de software para computación distribuida, basado en RPC.
1991	Aparición de Microsoft RPC basado en DCE para sistemas Windows.
1992	Aparición de DCOM (Microsoft) y CORBA (ORB) para la creación de componentes software distribuidos.
1997	Aparición de Java RMI en JDK 1.1
1998	Aparición de XML-RPC
1999	Aparición de SOAP 1.0, WSDL, UDDI
2000	Definición del REST
2012	Propuesta de GraphQL por Facebook
2015	Desarrollo de gRPC por Google

© JMA 2020. All rights reserved

Servicios SOAP

- SOAP es un protocolo de comunicación web altamente estandarizado basado en formatos de tipo texto en XML. Publicado por Microsoft en 1999, un año después de XML-RPC, SOAP heredó mucho de él.
- Basado en operaciones, de tipos texto, en formato XML y comunicaciones síncronas.
- Ventajas:
  - Independiente del lenguaje y la plataforma: La funcionalidad incorporada para crear servicios basados en web permite a SOAP manejar las comunicaciones y hacer que las respuestas sean independientes del lenguaje y la plataforma.
  - Vinculado a una variedad de protocolos de transporte: SOAP es flexible en términos de protocolos de transferencia para adaptarse a múltiples escenarios.
  - Manejo de errores incorporado: La especificación de la API SOAP permite devolver el mensaje Retry XML con el código de error y su explicación.
  - Varias extensiones de seguridad: Integrado con los protocolos WS-Security, SOAP cumple con una calidad de transacción de nivel empresarial. Proporciona privacidad e integridad dentro de las transacciones al tiempo que permite el cifrado a nivel de mensaje.

© JMA 2020. All rights reserved

Servicios SOAP

- Inconvenientes:
  - Solamente acepta formato XML: Los mensajes SOAP contienen una gran cantidad de metadatos y solo admiten estructuras XML detalladas para solicitudes y respuestas.
  - De peso pesado: Debido al gran tamaño de los archivos XML, los servicios SOAP requieren un gran ancho de banda hasta para la información mas nimia.
  - Conocimientos estrictamente especializados: La creación de servidores de API SOAP requiere un conocimiento profundo de todos los protocolos involucrados y sus reglas altamente restringidas o disponer de framework que simplifiquen su utilización que no están disponibles en todas las plataformas y lenguajes.
  - Actualización de mensajes tediosa: Al requerir un esfuerzo adicional para agregar o eliminar las propiedades del mensaje, el esquema SOAP rígido ralentiza la adopción.

© JMA 2020. All rights reserved

Servicios REST

- REpresentational State Transfer es un estilo arquitectónico autoexplicativo basado en el uso del protocolo HTTP e hipermedia y establece un conjunto de restricciones arquitectónicas y destinado a una amplia adopción. Definido en el 2000 por Roy Fielding, para no reinventar la rueda, se basa en aprovechar lo que ya estaba definido en el HTTP pero que no se utilizaba. RESTful hace referencia a un servicio web que implementa la arquitectura REST.
- Restringe la forma de usar de las URLs, los métodos (verbos) de HTTP, sus encabezados (Accept, Content-type, …) y sus códigos de estado.
- Basado en recursos, independiente de formato (texto o binario) y comunicaciones síncronas.
- Ventajas:
  - Soporte de múltiples formatos: La capacidad de admitir múltiples formatos (a menudo JSON y XML) para almacenar e intercambiar datos es una de las razones por las que REST es actualmente una opción predominante para crear APIs públicas.
  - Documentación mínima, basados en convenios y el descubrimiento hipermedia.
  - Mínima curva de aprendizaje: usa tecnologías ampliamente conocidas: HTTP, URL, MIME, …

© JMA 2020. All rights reserved

Servicios REST

- Inconvenientes:
  - Uso de HTTP: Aunque es la infraestructura mas ampliamente difundida y utilizada, está restringido a ella.
  - Grandes cargas útiles: REST devuelve una gran cantidad de metadatos enriquecidos para que el cliente pueda comprender todo lo necesario sobre el estado de la aplicación solo a partir de sus respuestas.
  - Sin estructura REST única: No existe una forma exacta y correcta de crear una API REST. Cómo modelar los recursos y qué recursos modelar dependerá de cada escenario. Esto hace que REST sea simple en teoría, pero difícil en la práctica.
  - Problemas de recuperación excesiva o insuficiente: Devolver todo suele conllevar demasiados datos y el convenio no estable mecanismos de filtrado, paginación o proyecciones. El uso de hipermedia para obtener datos relacionados requiere solicitudes adicionales y encadenadas.
  - Convenio genérico: En muchos casos no acorde con las reglas de negocio (create o replace, delete, …) lo que acaba requiriendo documentación.

© JMA 2020. All rights reserved

Servicios GraphQL

- GraphQL es una sintaxis que describe cómo obtener la descripción del modelo de datos y cómo realizar una solicitud de datos precisa, consultas y mutaciones, pensada para los modelos de datos con muchas entidades complejas que hacen referencia entre sí. Fue propuesta por Facebook en 2012, publicada en 2015 y posteriormente pasada a open source en 2018.
- Basado en consultas, en formato JSON y comunicaciones síncronas.
- Ventajas:
  - Un esquema de GraphQL establece una fuente única de información, ofrece una forma de unificar todo en un único servicio.
  - Las llamadas a GraphQL se gestionan mediante HTTP POST en un solo recorrido de ida y vuelta. Los clientes obtienen lo que solicitan sin que se genere una sobrecarga.
  - Los tipos de datos bien definidos reducen los problemas de comunicación entre el cliente y el servidor. Un cliente puede solicitar una lista de los tipos de datos disponibles y esto es ideal para la generación automática de documentación.
  - GraphQL permite que las APIs de las aplicaciones evolucionen sin afectar a las consultas actuales.
  - GraphQL no exige una arquitectura de aplicación específica. Puede incorporarse sobre una API de REST actual y funcionar con las herramientas de gestión de APIs actuales. Hay muchas extensiones open source de GraphQL que ofrecen características que no están disponibles con las APIs de REST.

© JMA 2020. All rights reserved

Servicios GraphQL

- Inconvenientes:
  - GraphQL intercambia complejidad por flexibilidad. Tener demasiados campos anidados en una solicitud puede provocar una sobrecarga del sistema. Además, delega gran parte del trabajo de las consultas de datos en el servidor, lo cual representa una mayor complejidad para los desarrolladores de servidores.
  - GraphQL no indica cómo almacenar los datos ni qué lenguaje de programación utilizar, requiere disponer de framework que simplifiquen su utilización que no están disponibles en todas las plataformas.
  - Como GraphQL no utiliza la semántica de almacenamiento en caché HTTP, requiere un esfuerzo de almacenamiento en caché personalizado.
  - GraphQL presenta una curva de aprendizaje elevada para desarrolladores que tienen experiencia con las APIs de REST.

© JMA 2020. All rights reserved

Servicios gRPC

- gRPC es un marco de llamada a procedimiento remoto (RPC) de alto rendimiento e independiente de lenguaje. Desarrollado por Google en el año 2015, y luego convertido en código abierto. Como GraphQL, es una especificación que se implementa en una variedad de lenguajes.
- Basado en contratos, en el formato binario Protocol Buffers y comunicaciones síncronas/asíncronas.
- Las principales ventajas de gRPC son:
  - Marco de RPC moderno, ligero y de alto rendimiento.
  - La especificación gRPC es preceptiva con respecto al formato que debe seguir un servicio gRPC, formaliza un contrato independiente de lenguaje.
  - Es compatible con el intercambio de datos bidireccional y asíncrono al estar basado en HTTP/2, así como con la compresión, multiplexación y streaming.
  - Uso reducido de red al usar un formato de mensaje binario eficaz que genera cargas de mensajes pequeñas.

© JMA 2020. All rights reserved

Servicios gRPC

- Inconvenientes:
  - Tanto el cliente como el servidor deben admitir la misma especificación de búfer de protocolo, requiere un estricto control de versiones. gRPC es un formato muy particular que proporciona una ejecución ultrarrápida a expensas de la flexibilidad.
  - Basado en HTTP/2 que no tiene soporte universal para interacciones cliente-servidor de cara al público general en Internet y una compatibilidad limitada con navegadores.
  - gRPC es una especificación, por lo que requiere disponer de framework que permitan su utilización tanto en el cliente como en le servidor y que no están disponibles en todas las plataformas y lenguajes.
  - El proceso de serialización/deserialización para su conversión a binario es costoso en términos de CPU.
  - gRPC presenta una curva de aprendizaje mas empinada que la de REST.

© JMA 2020. All rights reserved

Tecnología de clientes web (Navegadores)

- JavaScript Asíncrono + XML (AJAX) no es una tecnología por sí misma, es un término que describe un nuevo modo de utilizar conjuntamente varias tecnologías existentes. Esto incluye: HTML o XHTML, CSS, JavaScript, DOM, XML, XSLT, y lo más importante, el objeto XMLHttpRequest (fetch). Cuando estas tecnologías se combinan en un modelo AJAX, es posible lograr aplicaciones web capaces de actualizarse continuamente sin tener que volver a cargar la página completa.
- WebSockets es una tecnología avanzada que hace posible abrir una sesión de comunicación bidireccional entre el navegador y un servidor: se puede enviar mensajes a un servidor y recibir respuestas controladas por eventos sin tener que consultar al servidor para una respuesta.
- Server Sent Events (SSE) es una tecnología basada en streaming HTTP que pretenden estandarizar la comunicación COMET (long polling) en los navegadores cuyo flujo de comunicación sólo va desde el servidor hacia el cliente (no es bidireccional como los WebSockets). La idea consiste en que el cliente crea una conexión con el servidor una sola vez y este le va enviando información (eventos) al cliente cuando hay nuevos datos.
- ASP.NET Signalr es una biblioteca para desarrolladores de ASP.NET que simplifica el proceso de agregar funcionalidad web en tiempo real: la posibilidad de que el código de servidor inserte el contenido en los clientes conectados al instante a medida que esté disponible sin espera a nuevas solicitudes. Signalr usa WebSocket cuando está disponible o recurre las otras cuando es necesario.

© JMA 2020. All rights reserved

WebHooks

- Los webhooks son eventos que desencadenan acciones. Su nombre se debe a que funcionan como «ganchos» de los programas en Internet y casi siempre se utilizan para la comunicación entre sistemas. Son la manera más sencilla de obtener un aviso cuando algo ocurre en otro sistema y para el intercambio de datos entre aplicaciones web.
- Un webhook es una retro llamada HTTP, una solicitud HTTP POST insertada en una página web, que interviene cuando ocurre algo (una notificación de evento a través de HTTP POST).
- Los webhooks se utilizan para las notificaciones en tiempo real (con los datos del evento en JSON o XML) a una determinada dirección http:// o https://, que puede:
  - almacenar los datos del evento en JSON o XML
  - generar una respuesta que permita actualizarse al sistema donde se produce el evento
  - ejecutar un proceso en el sistema receptor del evento (Ej: enviar un correo electrónico)
- Los webhook están pensados para su utilización desde páginas web y sus diferentes consumidores: navegadores, correo electrónico, webapps, …
- Un ejemplo típico es su utilización en correos electrónico de marketing para notificar al servidor que debe enviar un nuevo correo electrónico porque el usuario a abierto el mensaje.
- Pueden considerarse una versión especializada y simplificada de los servicios REST (solo POST).

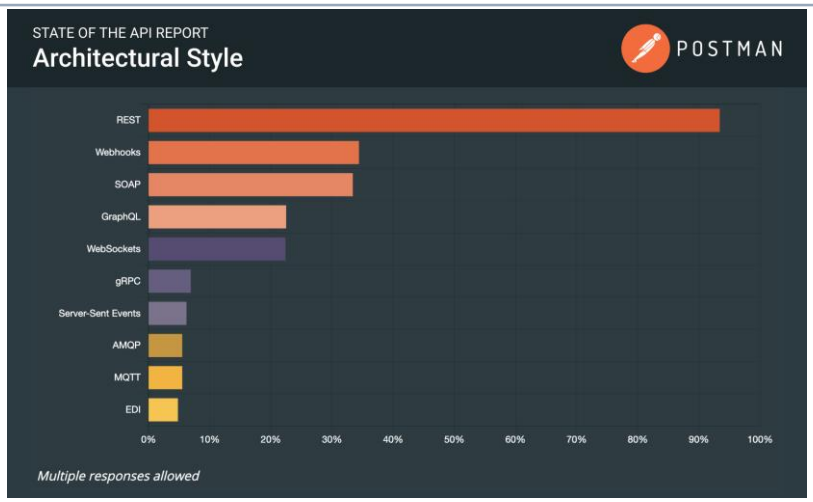
© JMA 2020. All rights reserved

Protocolos y Estándares

- Síncronos:
  - SOAP: Basado en operaciones, de tipos texto, en formato XML y comunicaciones síncronas.
  - REST: Basado en recursos, independiente de formato (texto/binario) y comunicaciones síncronas.
  - GraphQL: Basado en consultas, en formato JSON y comunicaciones síncronas.
  - gRPC: Basado en contratos, en el formato binario Protocol Buffers y comunicaciones síncronas/asíncronas.
- Asíncronos:
  - WebSockets: protocolo estándar abierto, elemental, texto y binario.
  - STOMP (Simple/Streaming Text Oriented Messaging Protocol): protocolo estándar abierto, basado en texto, soluciones simples y ligero.
  - AMQP (Advanced Message Queuing Protocol): protocolo estándar abierto, binario, encolamiento, P2P y Pub/Sub, exactitud y seguridad
  - MQTT (Message Queue Telemetry Transport): protocolo estándar abierto, binario, P2P, liviano, soluciones simples y seguridad
  - JMS (Java Message Service): API, binario Java, P2P y Pub/Sub

© JMA 2020. All rights reserved

Estilos arquitectónicos mas utilizados



© JMA 2020. All rights reserved

https://www.postman.com/state-of-api/api-technologies/#api-technologies

Patrón: Invocación a procedimiento remoto (RPI)

- Motivación:
  - Ha aplicado el patrón de Microservicios. Los servicios deben manejar las solicitudes de los clientes de la aplicación. Además, los servicios a veces deben colaborar para manejar esas solicitudes. Deben utilizar un protocolo de comunicación entre procesos.
- Intención:
  - ¿Cómo se van a comunicar los servicios?
- Requisitos:
  - Los servicios están escritos en diferentes tecnologías
  - Los clientes, a su vez, utilizan diferentes tecnologías. Hay aplicaciones web del lado del cliente (AJAX/WebSokets)
- Solución:
  - Utilizar RPI para la comunicación entre servicios. El cliente utiliza un protocolo basado en solicitud/respuesta para realizar solicitudes a un servicio. Los servicios públicos utilizaran REST.

© JMA 2020. All rights reserved

Patrón: Invocación a procedimiento remoto (RPI)

- Implementación:
  - Crear servicios REST con Spring Boot y Spring MVC con Spring HATEOAS, Spring Data Rest, …
- Consecuencias:
  - Este patrón tiene los siguientes beneficios:
    - Sencillo y familiar
    - El patrón solicitud/respuesta síncrona es fácil
    - Sistema más sencillo ya que no requiere componentes intermediarios.
  - Este patrón tiene los siguientes inconvenientes:
    - Por lo general, solo admite la solicitud/respuesta y no otros patrones de interacción, como notificaciones, solicitud/respuesta asíncrona, publicación/suscripción, publicación/respuesta asíncrona
    - Disponibilidad reducida ya que el cliente y el servicio deben estar disponibles durante la interacción
    - El cliente necesita descubrir ubicaciones de instancias de servicio.
- Patrones relacionados:
  - La mensajería es un patrón alternativo.
  - La configuración externalizada proporciona la ubicación de red (lógica).
  - Un cliente debe utilizar el descubrimiento del lado del cliente y el descubrimiento del lado del servidor para localizar una instancia de servicio
  - Normalmente, un cliente usará el patrón cortocircuito (interruptor automático) para mejorar la confiabilidad

© JMA 2020. All rights reserved

Patrón: Mensajería

- Motivación:
  - Ha aplicado el patrón de Microservicios. Los servicios deben manejar las solicitudes de los clientes de la aplicación. Además, los servicios a veces deben colaborar para manejar esas solicitudes. Deben utilizar un protocolo de comunicación entre procesos.
- Intención:
  - ¿Cómo se van a comunicar los servicios?
- Requisitos:
  - Los servicios están escritos en diferentes tecnologías
  - Los clientes, a su vez, utilizan diferentes tecnologías
- Solución:
  - Utilizar la mensajería asíncrona para la comunicación entre servicios. Servicios que se comunican mediante el intercambio de mensajes a través de canales de mensajería. El estándar AMQP (Advanced Message Queuing Protocol) es un protocolo estándar abierto en la capa de aplicaciones de un sistema de comunicación. Esta ampliamente difundido y soportado. Sus características son la orientación a mensajes, encolamiento ("queuing"), enrutamiento (tanto punto-a-punto como publicación-subscripción), exactitud y seguridad.

© JMA 2020. All rights reserved

Patrón: Mensajería

- Implementación:
  - Requiere un intermediario de mensajes: RabbitMQ, Apache Kafka
  - Crear servicios con Spring Boot con Spring for RabbitMQ, …
- Consecuencias:
  - Este patrón tiene los siguientes beneficios:
    - Bajo acoplamiento en tiempo de ejecución ya que desacopla el remitente del mensaje del consumidor
    - Disponibilidad mejorada desde que el intermediario de mensajes almacena los mensajes en búfer hasta que el consumidor puede procesarlos
    - Admite una variedad de patrones de comunicación que incluyen solicitud/respuesta síncrona y asíncrona, notificaciones, publicación/suscripción, publicación/respuesta asíncrona, etc.
  - Este patrón tiene los siguientes inconvenientes:
    - Mayor complejidad de los procesos asíncronos frente a los secuenciales síncronos.
    - Complejidad adicional, el intermediario de mensajes debe ser otro componente del sistema que debe instalarse y configurarse. También deberá ser replicado para disponibilidad y capacidad.
- Patrones relacionados:
  - La Invocación a procedimiento remoto (RPI) mensajería es un patrón alternativo.
  - La configuración externalizada proporciona los nombres de los canales de mensajes (lógicos) y la ubicación del intermediario de mensajes.
  - El patrón Saga y el patrón CQRS usan mensajes
  - El patrón de Bandeja de salida transaccional permite que los mensajes se envíen como parte de una transacción de base de datos

© JMA 2020. All rights reserved

Estilos de comunicación

MENSAJERÍA

© JMA 2020. All rights reserved

Mensajería y contextos de dominio

- En términos de integración, cuando se construyen estructuras de comunicación entre los diferentes procesos, es común ver productos y enfoques que ponen el énfasis de la inteligencia en el mecanismo de comunicación en sí.
- Normalmente los productos de ESB (Enterprise Service Bus) son un ejemplo donde generalmente se incluyen sofisticadas estructuras para el enrutado de mensajes, la coreografía, la transformación, la aplicación de reglas de negocio, etc.
- En términos de diseño, también es común ver que a la hora de definir dónde colocamos cierto código que representa reglas de negocio, a veces, decidimos hacerlo dentro del ESB por conveniencia o rapidez. Entonces, convertimos al ESB en un elemento de integración clave para el funcionamiento del proceso en su totalidad y, esa excesiva inteligencia puesta en un único componente (la tubería), torna al sistema en algo más frágil que antes.

© JMA 2020. All rights reserved

Mensajería y contextos de dominio

- Las aplicaciones creadas con microservicios pretenden ser tan disociadas y cohesivas como sean posible, ellas poseen su propia lógica de dominio y actúan más como filtros en el clásico sentido Unix:
  - recibe una solicitud, aplica la lógica apropiada y produce una respuesta.
  - estos pasos son coordinados utilizando protocolos REST simples en lugar de protocolos complejos WS-BPEL o la coordinación por una herramienta central.
- Los dos protocolos que se utilizan con mayor frecuencia son:
  - Petición/respuesta de HTTP con recursos API
  - Mensajería liviana, como puede ser RabbitMQ o ActiveMQ.
- Estos principios y protocolos hacen que los equipos de microservicios, a la hora de integrar servicios mas complejos, prefieran el concepto de coreografía, en lugar de orquestación.
- En orquestación, contamos con un cerebro central para orientar y conducir el proceso, al igual que el director de una orquesta.
- Con coreografía, le informamos a cada parte del sistema de su trabajo y se les deja trabajar en los detalles, como los bailarines en un ballet que se encuentran en su camino y reaccionan ante otros a su alrededor.

© JMA 2020. All rights reserved

Mensajería y contextos de dominio

- La desventaja del enfoque de orquestación es que el orquestador se puede convertir en una autoridad de gobierno central demasiado fuerte y pasar a ser un punto central donde toda la lógica gira alrededor de él.
- En cambio, con un enfoque coreografiado, podríamos tener un servicio emitiendo un mensaje asincrónico, los servicios interesados sólo se suscriben a esos eventos y reaccionan en consecuencia.
- Este enfoque es significativamente más desacoplado. Si algún nuevo servicio está interesado en recibir los mensajes, simplemente tiene que subscribirse a los eventos y hacer su trabajo cuando sea necesario.
- La desventaja de la coreografía es que la vista explícita del proceso de negocio del modelado de proceso ahora sólo se refleja de manera implícita en el sistema.
- Esto implica que se necesita de trabajo adicional para asegurarse de que alguien pueda controlar y realizar un seguimiento de que hayan sucedido las cosas correctas.
- Para solucionar este problema, normalmente es necesario crear un sistema de monitoreo que refleje explícitamente la vista del proceso de negocio del modelado de procesos, pero que a su vez, haga un seguimiento de lo que cada uno de los servicios realiza como entidad independiente que le permite ver excepciones mapeadas al flujo de proceso más explícito.

© JMA 2020. All rights reserved

Comandos vs Eventos

- Los mensajes se pueden clasificar en dos categorías principales:
  - Si el productor espera una acción del consumidor, ese mensaje es un comando.
  - Si el mensaje informa a los consumidor de que se ha realizado una acción, dicho mensaje es un evento.
- El productor envía un comando con la intención de que un consumidor realicen una operación dentro del ámbito de una transacción de negocio. Un comando es un mensaje de alto valor y debe entregarse al menos una vez. Si se pierde un comando, puede producirse un error en toda la transacción de negocio. Además, un comando no debe procesarse más de una vez. Si eso sucede, podría producirse una transacción errónea. Un cliente puede recibir pedidos o cargos duplicados. Los comandos se suelen usar para administrar el flujo de trabajo de una transacción de negocio de varios pasos. En función de la lógica de negocios, el productor puede esperar que el consumidor confirme el mensaje e informe de los resultados de la operación. En función de ese resultado, el productor puede elegir el plan de acción adecuado.
- Un evento es un tipo de mensaje que el productor genera para anunciar datos. El productor (conocido como publicador en este contexto) no espera que los eventos produzcan ninguna acción. Los consumidores interesados pueden suscribirse, escuchar eventos y tomar medidas en función de su escenario de consumo. Los eventos pueden tener varios suscriptores o ninguno. Dos suscriptores diferentes pueden reaccionar a un evento con diferentes acciones sin detectar al otro. El productor y el consumidor están acoplados de forma flexible y se administran de forma independiente. No se espera que el consumidor confirme el evento para el productor. Un consumidor que ya no está interesado en los eventos, puede finalizar la suscripción. El consumidor se quita de la canalización sin afectar al productor ni a la funcionalidad general del sistema.

© JMA 2020. All rights reserved

Colas de mensajes

- En determinadas ocasiones, los servicios deben integrarse con otros actores, componentes o sistemas internos y externos, siendo necesario aportar o recibir información de ellos. En la mayoría de los casos, estas comunicaciones tienen que estar permanentemente disponibles, ser rápidas, seguras, asíncronas y fiables entre otros requisitos.
- Las colas de mensajes (MQ) solucionan estas necesidades, actuando de intermediario entre emisores y destinatarios, o en un contexto más definido, productores y consumidores de mensajes. Se pueden usar para reducir las cargas y los tiempos de entrega por parte de los servicios, ya que las tareas, que normalmente tardarían bastante tiempo en procesarse, se pueden delegar a un tercero cuyo único trabajo es realizarlas.
- El uso de colas de mensajes también es bueno cuando se desea distribuir un mensaje a múltiples destinatarios. Además aportan otros beneficios como:
  - Garantía de entrega y orden: Los mensajes se consumen, en el mismo orden que se llegaron a la cola, y son consumidos una única vez
  - Redundancia: Las colas persisten los mensajes hasta que son procesados por completo
  - Desacoplamiento: Siendo capas intermedias de comunicación entre procesos, aportan la flexibilidad en la definición de arquitectura de cada uno de ellos de manera separada, siempre que se mantenga una interfaz común
  - Escalabilidad: Con más unidades de procesamiento, las colas balancean su respectiva carga

© JMA 2020. All rights reserved

RabbitMQ

- RabbitMQ (https://www.rabbitmq.com/) es el intermediario de mensajes de código abierto más ampliamente implementado. Dicho de forma simple, es un software donde se pueden definir colas de mensajes, las aplicaciones se pueden conectar a dichas colas y transferir/leer mensajes en ellas.
- RabbitMQ es ligero y fácil de implementar en las instalaciones y en la nube. Es compatible con múltiples protocolos de mensajería. RabbitMQ se puede implementar en configuraciones distribuidas y federadas para cumplir con los requisitos de alta disponibilidad y alta escalabilidad.
- RabbitMQ esta disponible en muchos sistemas operativos y entornos de nube, y proporciona una amplia gama de herramientas para desarrolladores en los lenguajes más populares.
- La arquitectura básica de una cola de mensajes es simple. Hay aplicaciones clientes, llamadas productores, que crean mensajes y los entregan al intermediario (la cola de mensajes). Otras aplicaciones, llamadas consumidores, se conectan a la cola y se suscriben a los mensajes que se procesarán. Un mensaje puede incluir cualquier tipo de información.

© JMA 2020. All rights reserved

RabbitMQ

- Un cliente puede ser productor, consumidor o productor y consumidor de mensajes. Los mensajes colocados en la cola se almacenan hasta que el consumidor los recupera y procesa.
- Los mensajes no se publican directamente en una cola, en lugar de eso, el productor envía mensajes a un exchange. Los exchanges son agentes de enrutamiento de mensajes, definidos por virtual host dentro de RabbitMQ. Un exchange es responsable del enrutamiento de los mensajes a las diferentes colas: acepta mensajes del productor y los dirige a colas de mensajes con ayuda de atributos de cabeceras, bindings y routing keys.
  - Un binding es un «enlace» que se configura para vincular una cola a un exchange
  - La routing key es un atributo del mensaje. El exchange podría usar esta clave para decidir cómo enrutar el mensaje a las colas (según el tipo de exchange)
  - Un virtual host es una división lógica de los componentes de servidor de RabbitMQ en la cual estos se agrupan para simplificar su administración y control.
- Los exchanges, las conexiones y las colas pueden configurarse con parámetros tales como durable, temporary y auto delete en el momento de su creación. Los exchanges declarados como durable sobrevivirán a los reinicios del servidor y durarán hasta que se eliminen explícitamente. Aquellos de tipo temporary existen hasta que RabbitMQ se cierre. Por último, los exchanges configurados como auto delete se eliminan una vez que el último objeto vinculado se ha liberado del exchange.

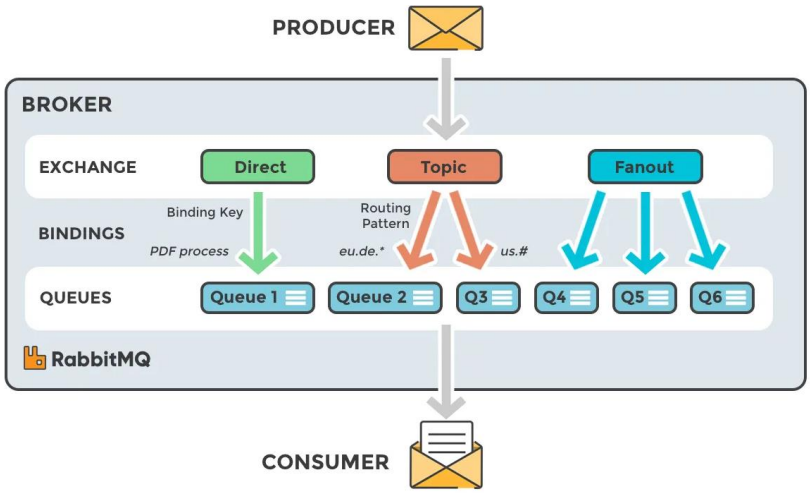
© JMA 2020. All rights reserved

RabbitMQ

- RabbitMQ ofrece varios tipos de exchanges, cada uno con un comportamiento de enrutamiento diferente:
  - Direct Exchange: Enruta los mensajes a las colas que tienen una clave de enrutamiento (routing key) idéntica a la clave de enrutamiento del mensaje. Es útil cuando se necesita una correspondencia precisa entre las claves de enrutamiento.
  - Fanout Exchange: Enruta los mensajes a todas las colas que están vinculadas al exchange, sin importar las claves de enrutamiento. Es útil cuando se desea enviar el mismo mensaje a múltiples destinatarios.
  - Topic Exchange: Enruta los mensajes a las colas basándose en patrones de enrutamiento definidos en las claves de enrutamiento. Los patrones pueden incluir comodines (* para una palabra, # para una o más palabras). Permite un enrutamiento más flexible en comparación con el Direct Exchange.
  - Headers Exchange: Enruta los mensajes según los encabezados específicos del mensaje, en lugar de las claves de enrutamiento. Puede ser útil cuando los criterios de enrutamiento son más complejos y se basan en metadatos del mensaje.

© JMA 2020. All rights reserved

RabbitMQ



© JMA 2020. All rights reserved

RabbitMQ

- Para garantizar que un mensaje nunca se pierda, RabbitMQ admite acuses de recibo (ACK o NACK) de mensajes. Existen dos modos principales de reconocimiento en RabbitMQ:
  - Modo Automático (Auto Acknowledgment): En este modo, una vez que RabbitMQ entrega un mensaje a un consumidor, lo considera entregado y lo quita de la cola inmediatamente, sin esperar un ACK. Esto significa que el consumidor no tiene que enviar un ACK manualmente. Sin embargo, esto también puede llevar a situaciones donde un mensaje se pierda si el consumidor falla antes de procesarlo.
  - Modo Manual (Manual Acknowledgment): En este modo, el consumidor debe enviar explícitamente un ACK al servidor una vez que ha procesado el mensaje y que puede eliminarlo. Esto permite un mayor control y garantiza que los mensajes no se pierdan fácilmente en caso de problemas en el consumidor. Con NACK le informa que debe reintegrarlo a la cola. Es un error común, con consecuencias graves, pasar por alto el ACK, dado que los mensajes se volverán a entregar cuando el consumidor se cierre (reintegra a la cola los mensajes no confirmados). Se puede configurar que si un consumidor no envía un ACK dentro de un cierto período de tiempo, RabbitMQ considere que el mensaje no ha sido entregado y lo vuelva a encolar.
- Se pueden configurar tiempos de vida para los mensajes para garantizar que los mensajes no queden encolados indefinidamente.
- Por defecto RabbitMQ enviará cada mensaje al siguiente consumidor, en secuencia (Round-Robin). Se puede configurar para que no entregue más de un mensaje a un consumidor a la vez: no enviara un nuevo mensaje a un consumidor hasta que haya procesado y reconocido el anterior.

© JMA 2020. All rights reserved

RabbitMQ

- RabbitMQ es un sistema de mensajería que maneja mensajes en forma de datos serializados y no impone restricciones específicas sobre los tipos de datos que pueden ser enviados en los mensajes, ya que los mensajes son simplemente secuencias de bytes. Algunos ejemplos de formatos de datos que se pueden enviar son:
  - Texto: Cadenas de texto en mensajes para transmitir información legible por humanos.
  - JSON o XML: Es común enviar datos estructurados en formato JSON o XML para intercambiar información entre sistemas.
  - Datos Binarios: Se pueden enviar archivos y datos binarios (imágenes, videos, documentos, etc.) codificados en base64 u otros formatos en los mensajes.
  - Objetos Serializados: En lenguajes que soportan serialización de objetos (como Java o C# con objetos serializables), se pueden enviar objetos serializados como mensajes.
  - Formatos Propios: Definiendo cómo los datos serán codificados y decodificados por los productores y consumidores.
- Es importante tener en cuenta que tanto el productor como el consumidor deben estar de acuerdo en el formato de los datos que se están intercambiando.
- Es recomendable seleccionar un formato que sea eficiente para el uso de los recursos y que permita una fácil interpretación por parte de los sistemas que envían y reciben los mensajes.

© JMA 2020. All rights reserved

Instalación

- Descargar la última versión estable para Windows desde:
  - https://www.rabbitmq.com/download.html
- Para arrancar el servicio:
  - rabbitmqctl.bat start
- Para parar el servicio:
  - rabbitmqctl.bat stop
- Instalación del complemento de administración:
  - rabbitmq-plugins enable rabbitmq_management
- Para crear un contenedor:
  - docker run -d --name rabbitmq -p 4369:4369 -p 5671:5671 -p 5672:5672 -p 15671:15671 -p 15672:15672 -p 25672:25672 -e RABBITMQ_DEFAULT_USER=admin -e RABBITMQ_DEFAULT_PASS=curso rabbitmq:management-alpine
- Para acceder al complemento de administración:
  - http://localhost:15672
  - El agente crea un usuario “guest” con contraseña “guest”.

© JMA 2020. All rights reserved

RabbitMQ: Spring Boot

- Añadir al proyecto:
  - Messaging > Spring for RabbitMQ
- Configurar:

```
spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest
#spring.rabbitmq.template.retry.enabled=true
#spring.rabbitmq.template.retry.initial-interval=2s
```

- AmqpTemplate y AmqpAdmin se configuran de forma automáticamente.
- Anotaciones:
  - @Queue: definición de una cola.
  - @Exchange: definición de un enrutador
  - @QueueBinding: Define una cola, el intercambio al que debe vincularse y una clave de enlace opcional, utilizada con @RabbitListener.
  - @RabbitListener: indica que es un método de escucha de una cola.

© JMA 2020. All rights reserved

Manejar una cola

- Crear cola si no existe:

```
@Bean
public Queue myQueue() {
        return new Queue("mi-cola");
}
```

- Enviar un mensaje:

```
@Autowired
private AmqpTemplate amqp;

public void send(String cola, MessageDTO outMsg) {
        amqp.convertAndSend(cola, outMsg);
}
```

- Recibir mensajes:

```
@RabbitListener(queues = "mi-cola")
public void recive(MessageDTO inMsg) {
        // Procesar el mensaje recibido: inMsg
}
```

© JMA 2020. All rights reserved

Formato del Mensaje

- En la configuración, crear el formateador de los mensajes:

```
@Bean
public MessageConverter jsonConverter() () {
    return new Jackson2JsonMessageConverter();
}
```

- En la configuración, crear la versión personalizada del RabbitTemplate con el formateador recién creado:

```
@Bean
public RabbitTemplate rabbitTemplate(final ConnectionFactory connectionFactory) {
    RabbitTemplate rabbitTemplate = new RabbitTemplate(connectionFactory);
    rabbitTemplate.setMessageConverter(jsonConverter());
    return rabbitTemplate;
}
```

© JMA 2020. All rights reserved

RabbitMQ: .NET

- Añadir al proyecto:
  - dotnet add package RabbitMQ.Client
- Para crear una conexión y abrir un canal:

```
using RabbitMQ.Client;

var factory = new ConnectionFactory {
   HostName = "localhost", Port = 5672,
   UserName = "admin", Password = "curso",
   ClientProvidedName = "Async.Amqp.Receptor"
};
var connection = factory.CreateConnection();
var channel = connection.CreateModel();
```

© JMA 2020. All rights reserved

Manejar una cola (.Net)

- Crear cola si no existe:

```
channel.QueueDeclare(queue: "demo.saludos",
                     durable: true,
                     exclusive: false,
                     autoDelete: false,
                     arguments: null);
```

- Enviar un mensaje:

```
var body = Encoding.UTF8.GetBytes(message);
channel.BasicPublish(exchange: exchange,
                     routingKey: routingKey,
                     basicProperties: props,
                     body: body);
```

© JMA 2020. All rights reserved

Manejar una cola (.Net)

- Recibir mensajes:

```
var consumer = new EventingBasicConsumer(channel);
consumer.Received += (model, ev) => {
   var canal = (model as EventingBasicConsumer).Model;
   var body = ev.Body.ToArray();
   var message = Encoding.UTF8.GetString(body);
   Console.WriteLine($" [x] Received {message}");
   canal.BasicAck(ev.DeliveryTag, false);
};
channel.BasicConsume(queue: queueName,
                     autoAck: false,
                     consumer: consumer);
```

© JMA 2020. All rights reserved

Eventos

- Un evento es una notificación que indica un cambio en un entorno, una aplicación o un servicio.
- En una arquitectura basada en eventos, los productores de eventos generan un flujo de eventos que los consumidores de eventos escuchan. Los productores están desconectados entre sí y de los consumidores, no saben si los consumidores están escuchando. A diferencia de los mensajes, un evento puede ser consumido por múltiples consumidores o por ninguno. Dado que un evento puede no ser consumido, la carga útil suele estar reducida al máximo en comparación con los mensajes.
- La transmisión de eventos (event streaming) es la práctica de:
  - capturar datos en tiempo real de las fuentes de eventos (como bases de datos, sensores, dispositivos móviles, IoT, servicios en la nube y aplicaciones de software) en forma de flujos de eventos
  - almacenar estos flujos de eventos de forma duradera para su posterior recuperación
  - manipular, procesar y reaccionar a los flujos de eventos en tiempo real y retrospectivamente
  - enrutar los flujos de eventos a diferentes tecnologías de destino según sea necesario.
- La transmisión de eventos garantiza un flujo continuo y una interpretación de los datos para que la información correcta esté en el lugar correcto y en el momento correcto.
- Los eventos se entregan casi en tiempo real, de modo que los consumidores pueden responder inmediatamente a los eventos cuando se producen. Un consumidor puede unirse en cualquier momento y capturar tantos eventos anteriores como desee.

© JMA 2020. All rights reserved

Apache Kafka

- Apache Kafka es una popular plataforma de streaming de eventos y de código abierto, que sirve para recoger, procesar y almacenar datos de eventos de streaming o datos sin principio ni final concretos. Kafka posibilita aplicaciones distribuidas que pueden escalar para gestionar miles de millones de eventos de streaming cada minuto y casi en tiempo real.
- Al estar diseñado específicamente para la transmisión de registros en tiempo real, Apache Kafka es ideal para las aplicaciones que requieren:
  - Intercambio de datos fiables entre diferentes componentes
  - La capacidad de dividir las cargas de trabajo de mensajería a medida que cambian los requisitos de la aplicación
  - Transmisión en tiempo real para el procesamiento de datos
  - Soporte nativo para la reproducción de datos/mensajes

© JMA 2020. All rights reserved

Instalación

- Hay varias opciones disponibles para instalar y configurar el entorno. Se pueden instalar los componentes necesarios en la máquina host de forma tradicional, pero es mas conveniente utilizar contenedores Docker dado que tiene la ventaja de ser limpia, independiente del sistema y fácil de configurar.
- Los componentes imprescindibles para operar con Apache Kafka son:
  - Kafka Brokers: Cada uno de los servicios intermediarios de Kafka que conforman el clúster. Almacenan y distribuyen los datos, que se organizan en topics.
  - Zookeeper: Gestiona los brokers de Kafka y les envía notificaciones en caso de cambio como creación de topics, borrado de topics, caída de broker, recuperación de broker …
  - UI for Apache Kafka (opcional): Es una interfaz web de usuario gratuita y de código abierto para monitorizar y administrar clústeres de Apache Kafka.

© JMA 2020. All rights reserved

Docker Compose

```
services:
  zookeeper:
    image: confluentinc/cp-zookeeper:latest
    container_name: zookeeper
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
      ZOOKEEPER_TICK_TIME: 2000
    ports:
      - 2181:2181

  kafka:
    image: confluentinc/cp-kafka:latest
    container_name: kafka
    depends_on:
      - zookeeper
    ports:
      - 9092:9092
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS:
PLAINTEXT://kafka:29092,PLAINTEXT_HOST://localhost:9092
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
      KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

  kafka-ui:
    image: provectuslabs/kafka-ui
    container_name: kafka-ui
    depends_on:
      - kafka
    ports:
      - 9091:8080
    environment:
      - KAFKA_CLUSTERS_0_NAME=local
      - KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS=kafka:29092
      - KAFKA_CLUSTERS_0_ZOOKEEPER=localhost:2181
```

© JMA 2020. All rights reserved

Conceptos

- Un **evento** registra el hecho de que "algo sucedió" en el mundo o en el negocio. También se le llama registro o mensaje en la documentación. Cuando se leen o escriben datos en Kafka, se hace en forma de eventos. Conceptualmente, un evento tiene una clave, un valor, una marca de tiempo y encabezados de metadatos opcionales.
- Los **productores** son aquellas aplicaciones cliente que publican (escriben) eventos en Kafka, y los **consumidores** son los que se suscriben (leen y procesan) estos eventos. Los consumidores se pueden agrupar para que solo un miembro del grupo reciba y procese el evento. En Kafka, los productores y los consumidores están totalmente desvinculados y son independientes entre sí, lo cual es un elemento de diseño clave para lograr la alta escalabilidad por la que Kafka es conocido.
- Los eventos se organizan y almacenan de forma duradera en temas con nombre (**topics**), similar a las colas de mensajes. Los temas en Kafka son siempre multiproductor y multisuscriptor: un tema puede tener cero, uno o muchos productores que escriben eventos en él, así como cero, uno o muchos consumidores que se suscriben a estos eventos. Los eventos de un tema se pueden leer tantas veces como sea necesario; a diferencia de los sistemas de mensajería tradicionales, los eventos no se eliminan después del consumo. En su lugar, se define a través de la configuración del tema durante cuánto tiempo Kafka debe retener los eventos, después del cual se descartarán los eventos antiguos. El rendimiento de Kafka es efectivamente constante con respecto al tamaño de los datos, por lo que almacenar datos durante mucho tiempo puede estar perfectamente bien.

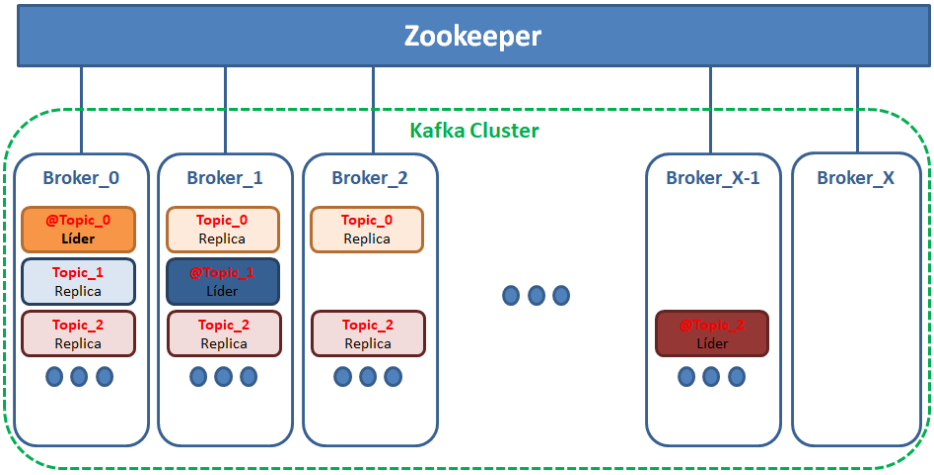
© JMA 2020. All rights reserved

Conceptos

- Los temas están **particionados**, lo que significa que un tema se distribuye en una o varias "agrupaciones" ubicadas en diferentes intermediarios (brokers) de Kafka. Esta ubicación distribuida de los datos es muy importante para la escalabilidad porque permite que las aplicaciones de los clientes lean y escriban los datos desde/hacia muchos intermediarios al mismo tiempo. Cuando se publica un nuevo evento en un tema, en realidad se agrega a una de las particiones del tema. Los eventos con la misma clave de evento se escriben en la misma partición, y Kafka garantiza que cualquier consumidor de una partición de un tema determinado siempre leerá los eventos de esa partición exactamente en el mismo orden en que se escribieron. A cada mensaje se le asigna un número incremental u **offset** que identifica cada mensaje dentro de la partición, la diferencia entre el offset producido y el offset consumido se denomina **lag**, con una valor cercano a cero significa que el consumidor está procesando los mensajes sin retardo y con valores altos y que se incrementan significa que el consumidor no está siendo capaz de procesar el volumen de mensajes producidos.
- Para que los datos sean tolerantes a fallos y estén altamente disponibles, cada tema se puede **replicar**, incluso entre regiones geográficas o centros de datos, de modo que siempre haya varios intermediarios que tengan una copia de los datos en caso de que las cosas salgan mal, para hacer el mantenimiento del intermediario o cualquier otra contingencia. Una configuración de producción común es un factor de replicación de 3, es decir, siempre habrá tres copias de los datos. Esta replicación se realiza a nivel de particiones de tema.

© JMA 2020. All rights reserved

Conceptos



© JMA 2020. All rights reserved

Conceptos

- Los **grupos de consumidores**, identificados por un Group ID común, permiten la escalabilidad horizontal y el paralelismo en el procesamiento de datos. Cada partición de un tópico solo puede ser consumida por un consumidor dentro de un grupo en un momento dado, si se desconecta salta al siguiente. Esto permite que múltiples consumidores dentro de un grupo trabajen en paralelo y procesen los datos de manera eficiente.
- Apache Kafka tiene un modelo de entrega de mensajes garantizado llamado "**acknowledgment**" (reconocimiento), denominado "ack", que permite a los productores saber si un mensaje ha sido entregado correctamente a los brokers y, por lo tanto, está disponible para ser consumido por los consumidores. Kafka garantiza que el mensaje ha sido entregado según el nivel de ack especificado pero no garantiza que el mensaje haya sido consumido por un consumidor: la confirmación de entrega se refiere a la disponibilidad del mensaje en los brokers para su consumo. Para verificar si un mensaje ha sido consumido, los consumidores pueden confirmar su procesamiento por separado y luego avanzar el offset de la partición para indicar que han leído y manejado ese mensaje.
- Kafka no impone ninguna restricción en el **contenido** de los mensajes, los trata simplemente como una cantidad de bytes ya sean de texto, comprimidos o cifrados.

© JMA 2020. All rights reserved

API de Kafka

- **Admin API**: para administrar e inspeccionar temas, intermediarios y otros objetos de Kafka.
- **Producer API**: para publicar (escribir) un flujo de eventos en uno o más temas de Kafka.
- **Consumer API**: para suscribirse a (leer) uno o más temas y procesar el flujo de eventos producidos para ellos.
- **Kafka Streams API**: para implementar microservicios y aplicaciones de procesamiento de flujo. Proporciona funciones de nivel superior para procesar flujos de eventos, incluidas transformaciones, operaciones con estado como agregaciones y uniones, ventanas, procesamiento basado en el tiempo del evento y más. La entrada se lee de uno o más temas para generar una salida a uno o más temas, transformando efectivamente los flujos de entrada en flujos de salida.
- **Kafka Connect API**: para crear y ejecutar conectores de importación/exportación de datos reutilizables que consumen (leen) o producen (escriben) flujos de eventos desde y hacia sistemas y aplicaciones externos para que puedan integrarse con Kafka, hay cientos de conectores listos para usar.
- Herramientas de línea de comandos para tareas de gestión y administración.

© JMA 2020. All rights reserved

Apache Kafka: Spring Boot

- Añadir al proyecto:
  - Messaging > Spring for Apache Kafka
- Configurar:

```
spring.kafka.bootstrap-servers=${KAFKA_BROKER:http://localhost:9092}
```

- KafkaTemplate<String, String> se configuran de forma automáticamente.

```
@Autowired
private KafkaTemplate<String, String> kafkaTemplate;
```

- Para crear un tema si no existe:

```
@Bean
public NewTopic topicLocation() {
    return new NewTopic("mi-tema", 1, (short) 1);
}
```

© JMA 2020. All rights reserved

Enviar un evento

```
// la clave (key) es opcional
public void sendMessage(String topic, String key, String message) {
        kafkaTemplate.send(topic, key, message)
            .thenAccept(result -> LOG.info(String.format("TOPIC: %s, KEY: %s,
                MESSAGE: %s, OFFSET: %s", topic, key, message,
                result.getRecordMetadata().offset())))
            .exceptionally(ex -> {
                LOG.error(String.format("TOPIC: %s, KEY: %s, MESSAGE: %s, ERROR:
                    %s", topic, key, message, ex.getMessage()));
                return null;
            });
}
```

© JMA 2020. All rights reserved

Recibir eventos

- Recibir los mensajes de los eventos:

```
@KafkaListener(topics = "${topic.name}", groupId = "nombre-consumidor")
public void listenTopic(String message) {
    LOG.info("Mensaje: " + message);
}
```

- Extraer los metadatos de los eventos:

```
@KafkaListener(topics = "${topic.name}", topicPattern = "${topic.name}", groupId = "nombre-
    consumidor")
public void listenWithHeaders(@Header(KafkaHeaders.RECEIVED_KEY) String key, @Payload String
    message, @Header(KafkaHeaders.OFFSET) String offset) {
    LOG.info(String.format("KEY: %s, MESSAGE: %s, OFFSET: %s", key, message, offset));
}
```

- Para recibir todos los eventos anteriores del tema:
  - spring.kafka.consumer.auto-offset-reset=earliest

© JMA 2020. All rights reserved

Apache Kafka: .NET

- Añadir al proyecto:
  - dotnet add package Confluent.Kafka
- Para crear un tema si no existe:

```
using Confluent.Kafka;
using Confluent.Kafka.Admin;

using(var adminClient = new AdminClientBuilder(new AdminClientConfig { BootstrapServers = brokerList
}).Build()) {
  try {
    await adminClient.CreateTopicsAsync([
      new TopicSpecification { Name = topicName, ReplicationFactor = 1, NumPartitions = 1 }
    ]);
  } catch(CreateTopicsException e) {
    if(!e.Results[0].Error.Reason.Contains("already exists"))
      Console.WriteLine($"An error occurred creating topic {e.Results[0].Topic}: {e.Results[0].Error.Reason}");
  }
}
```

© JMA 2020. All rights reserved

Enviar un evento

```
// la clave (key) es opcional
var config = new ProducerConfig { BootstrapServers = brokerList };
using(var producer = new ProducerBuilder<string, string>(config).Build()) {
   for(int i = 1; i <= 100; i++) {
      try {
         var delay = Random.Shared.Next(500, 1500);
         Thread.Sleep(delay);
         var message =new Message<string, string> { Key = "Emisor-01", Value = $"Evento {i}"};
         var deliveryReport = await producer.ProduceAsync(topicName, message);
         Console.WriteLine($"entrega ({delay / 1000.0}s): {deliveryReport.Offset} {message.Value}");
      } catch(ProduceException<string, string> e) {
         Console.WriteLine($"failed to deliver message: {e.Message} [{e.Error.Code}]");
      }
   }
}
```

© JMA 2020. All rights reserved

Recibir eventos

```
CancellationTokenSource cts = new CancellationTokenSource();
Console.CancelKeyPress += (_, e) => { e.Cancel = true;  cts.Cancel(); };
var cancellationToken = cts.Token;
var config = new ConsumerConfig { BootstrapServers = brokerList, GroupId = groupName, EnableAutoOffsetStore = false, EnableAutoCommit = true };
using(var consumer = new ConsumerBuilder<string, string >(config).SetErrorHandler((_, e) => Console.WriteLine($"Error: {e.Reason}")).Build()) {
   consumer.Subscribe(topicName);
   try {
      while(true) {
         try {
            var consumeResult = consumer.Consume(cancellationToken);
            if(consumeResult.IsPartitionEOF) {
               Console.WriteLine($"Reached end of topic {consumeResult.Topic}, partition {consumeResult.Partition}, offset {consumeResult.Offset}.");
               continue;
            }
            Console.WriteLine($"Received {consumeResult.Offset}: {consumeResult.Message.Key} - {consumeResult.Message.Value}");
            try { consumer.StoreOffset(consumeResult); } catch(KafkaException e) { Console.WriteLine($"Store Offset error: {e.Error.Reason}"); }
         } catch(ConsumeException e) {  Console.WriteLine($"Consume error: {e.Error.Reason}"); }
      }
   } catch(OperationCanceledException) {
      Console.WriteLine("Closing consumer.");
      consumer.Close();
   }
}
```

© JMA 2020. All rights reserved

ActiveMQ (JMS)

- Apache ActiveMQ es un intermediario de mensajes basado en Java, multiprotocolo y de código abierto, que implementa plenamente la especificación de Java Message Service 1.1 (JMS).
- Es compatible con los protocolos estándar de la industria (AMQP, STOMP sobre websockets, MQTT y JMS) para que los usuarios obtengan los beneficios de las opciones del cliente en una amplia gama de lenguajes y plataformas (JavaScript, C, C++, Python, .Net y más).
- Actualmente hay dos "sabores" de ActiveMQ disponibles: el conocido como broker "clásico" y el broker de "próxima generación" cuyo nombre en código es Artemis. Una vez que Artemis alcance un nivel suficiente de paridad de funciones con el broker "clásico", se convertirá en la versión principal de ActiveMQ.
- El broker "Classic" ofrece características empresariales tales como JMS 1.1 con implementación de cliente completa que incluye JNDI, alta disponibilidad mediante almacenamiento compartido, modelo familiar de direccionamiento basado en JMS, "Red de intermediarios" para la distribución de carga y KahaDB o JDBC para persistencia.
- Artemis ofrecerá JMS 1.1 y 2.0 + Jakarta Messaging 2.0 y 3.0 con implementaciones de cliente completas (incluido JNDI), alta disponibilidad mediante almacenamiento compartido o replicación de red, modelo simple y potente de direccionamiento agnóstico de protocolo, agrupación flexible para distribuir la carga, implementaciones avanzadas del diario para persistencia de baja latencia (así como JDBC), duplicación asíncrona para recuperación ante desastres y equilibrio de carga basado en datos.
- ActiveMQ "Artemis" dispondrá de una alta paridad de características con ActiveMQ "Classic" para facilitar la migración.

© JMA 2020. All rights reserved

Instalación

- Descargar la última versión estable para Windows desde:
  - https://activemq.apache.org/components/classic/download/
- Extraer los archivos del archivo ZIP en el directorio de instalación.
- Para arrancar el servicio, desde una ventana de consola, cambiar al directorio de instalación y ejecutar:
  - bin\activemq start
- Los directorios de trabajo se crean en relación con el directorio actual. Para crear directorios de trabajo en el lugar adecuado, ActiveMQ debe iniciarse desde su directorio de inicio/instalación.
- Para acceder al complemento de administración, desde el navegador:
  - URL: http://127.0.0.1:8161/admin/
    - Iniciar sesión: admin
    - Contraseña: admin
  - Ir a "Queues"
  - Agregar un nombre de cola y hacer clic en crear
  - Enviar mensaje de prueba haciendo clic en "Enviar a"
  - Consultar con atom o rss

© JMA 2020. All rights reserved

Protocolos

- Apache ActiveMQ es un intermediario de mensajes que admite múltiples protocolos de nivel de conexión para una máxima interoperabilidad.
  - Core-UI (port: 8161)
  - REST (port: 8161)
  - RSS y Atom (port: 8161)
  - AMQP (port: 5672)
  - MQTT (port: 1883)
  - WebSocket (port: 61614)
  - Stomp (port: 61613)
  - JMS (port: 61616)
  - OpenWire (port: 61616)
  - ~~XMPP (port: 61222)~~

© JMA 2020. All rights reserved

Conceptos

- Soporta dos modelos/dominios de mensajería:
  - Punto a Punto (Queues), en la que un mensaje se consume por un único consumidor.
  - Publicación/Subscripción (Topics), en la que un mensaje se consume por muchos consumidores.
- Soporta el modelo de mensaje unificado de JMS que especifica que todos los mensajes deben contar con cabeceras, propiedades y cuerpo. Las cabeceras ofrecen metadatos sobre el mensaje utilizado por ambos clientes y el proveedor JMS. Las propiedades son campos opcionales dentro del mensaje para añadir información adicional al mensaje. El cuerpo puede contener tanto texto como datos binarios mediante los diferentes tipos de mensajes.
- JMS define seis tipos de cuerpo para los mensajes, también conocidos tipos de mensajes:
  - Message: Tipo de mensaje base. Se utiliza para enviar un mensaje sin cuerpo, solo cabeceras y propiedades. Normalmente se utiliza para notificación simple de eventos.
  - MapMessage: Compuesto de un conjunto de pares {nombre,valor}. El tipo de los nombres es String, y los valores tipos primitivos Java. A los nombres (que no están ordenados) podemos acceder de forma secuencial mediante un enumerador, o por acceso directo por nombre.
  - BytesMessage: Contiene un array de bytes sin interpretar. Se utiliza para hacer coincidir el cuerpo con un formato de mensaje existente (legacy).
  - StreamMessage: El cuerpo es un flujo de tipos primitivos Java, cuya lectura y escritura se realiza de modo secuencial.
  - TextMessage: Un mensaje cuya carga es un String. Se suele utilizar para enviar texto simple y datos XML o JSON.
  - ObjectMessage: La carga es un objeto Java Serializable. Normalmente se utiliza para trabajar con objetos Java complejos.

© JMA 2020. All rights reserved

ActiveMQ: Spring Boot

- Para añadir al proyecto ActiveMQ 5:
  - Messaging > Spring for Apache ActiveMQ 5
- Configurar:

```
#spring.activemq.broker-url=tcp://localhost:61616
#spring.activemq.user=admin
#spring.activemq.password=admin
spring.activemq.in-memory=false
```

- Para añadir al proyecto ActiveMQ Artemis:
  - Messaging > Spring for Apache ActiveMQ Artemis
- Configurar:

```
#spring.artemis.mode=native
spring.artemis.host=localhost
spring.artemis.port=61616
#spring.artemis.user=admin
#spring.artemis.password=admin
```

© JMA 2020. All rights reserved

Template y Anotaciones

- Spring proporciona un marco de integración de JMS que simplifica el uso del API de JMS de la misma manera que lo hace para JDBC.
- JMS se puede dividir aproximadamente en dos áreas de funcionalidad, a saber, la producción y el consumo de mensajes.
- La clase JmsTemplate se utiliza para la producción de mensajes y la recepción de mensajes sincrónicos.
- Para una recepción asincrónica similar al estilo de bean controlado por mensajes de Jakarta EE, Spring proporciona una serie de contenedores de escucha de mensajes que puede usar para crear POJO controlados por mensajes (MDP). Spring también proporciona una forma declarativa con anotaciones de crear oyentes de mensajes.

© JMA 2020. All rights reserved

Queues: enviar y recibir

- Enviar un mensaje:

```
@Autowired
JmsTemplate jms;

@GetMapping(path = "/saludo/{nombre}")
public String saluda(@PathVariable String nombre) {
        String msg = "Hola " + nombre;
        jms.convertAndSend("saludos", new MessageDTO(msg, origen));
        return "SEND COLA: " + msg;
}
```

- Recibir mensajes:

```
@JmsListener(destination = "saludos")
public void listenQueue(MessageDTO in) {
        Store.addQueue(new Message(in));
        log.warn("MENSAJE RECIBIDO: " + in);
}
```

© JMA 2020. All rights reserved

Topics: enviar y recibir

- Se puede establecer la Publicación/Subscripción como mecanismo por defecto en application.properties:

```
spring.jms.pub-sub-domain=true
```

- En caso de convivir los dos sistemas hay que crear uno o varios contenedores personalizados para asignarlos a los listener:

```
@Bean
JmsListenerContainerFactory<?> myPubSubFactory(ConnectionFactory connectionFactory,
            DefaultJmsListenerContainerFactoryConfigurer configurer) {
    DefaultJmsListenerContainerFactory factory = new DefaultJmsListenerContainerFactory();
    configurer.configure(factory, connectionFactory);
    factory.setPubSubDomain(true);
    return factory;
}
```

© JMA 2020. All rights reserved

Topics: enviar y recibir

- Enviar un mensaje:

```
@Autowired
JmsTemplate jms;
@GetMapping(path = "/despedida/{nombre}")
public String despedida(@PathVariable String nombre) {
        String msg = "Adios " + nombre;
        jms.setPubSubDomain(true);  // Si por defecto spring.jms.pub-sub-domain=false
        jms.convertAndSend("despedidas", new MessageDTO(msg, origen));
        return "SEND TEMA: " + msg;
}
```

- Recibir mensajes:

```
@JmsListener(destination = "despedidas", containerFactory = "myPubSubFactory")
public void listen(MessageDTO in) {
        Store.addTopic(new Message(in));
        log.warn("TEMA RECIBIDO: " + in);
}
```

© JMA 2020. All rights reserved

Formato del Mensaje

- Los objetos ObjectMessage dependen de la serialización de Java de la carga útil del objeto marshal/unmarshal. Este proceso generalmente se considera inseguro ya que una carga útil maliciosa puede explotar el sistema host. Es por eso que a partir de las versiones 5.12.2 y 5.13.0 , ActiveMQ obliga a incluir explícitamente en la lista blanca de los paquetes que se pueden intercambiar.

  -Dorg.apache.activemq.SERIALIZABLE_PACKAGES=*

- Para facilitar el envío de objetos del modelo de dominio, JmsTemplate tiene varios métodos de envío que toman un objeto Java como argumento para el contenido de datos de un mensaje. Los métodos sobrecargados convertAndSend() y receiveAndConvert() delegan el proceso de conversión a una instancia de la interfaz MessageConverter. Esta interfaz define un contrato simple para convertir entre objetos Java y mensajes JMS. La implementación predeterminada (SimpleMessageConverter) admite la conversión entre String a TextMessage, byte[] a BytesMessage y java.util.Map a MapMessage.

© JMA 2020. All rights reserved

Formato del Mensaje

- Los objetos ObjectMessage dependen de la serialización de Java de la carga útil del objeto marshal/unmarshal. Este proceso generalmente se considera inseguro ya que una carga útil maliciosa puede explotar el sistema host. Es por eso que a partir de las versiones 5.12.2 y 5.13.0 , ActiveMQ obliga a incluir explícitamente en la lista blanca de los paquetes que se pueden intercambiar.

  -Dorg.apache.activemq.SERIALIZABLE_PACKAGES=*
- Para facilitar el envío de objetos del modelo de dominio, JmsTemplate tiene varios métodos de envío que toman un objeto Java como argumento para el contenido de datos de un mensaje. Los métodos sobrecargados convertAndSend() y receiveAndConvert() delegan el proceso de conversión a una instancia de la interfaz MessageConverter. Esta interfaz define un contrato simple para convertir entre objetos Java y mensajes JMS. La implementación predeterminada (SimpleMessageConverter) admite la conversión entre String a TextMessage, byte[] a BytesMessage y java.util.Map a MapMessage.

© JMA 2020. All rights reserved

Formato del Mensaje

- Para establecer un conversor predeterminado:

```
@Bean // Serialize message content to json using TextMessage
MessageConverter jacksonJmsMessageConverter() {
        MappingJackson2MessageConverter converter = new MappingJackson2MessageConverter();
        converter.setTargetType(MessageType.TEXT);
        converter.setTypeIdPropertyName("_type");
        return converter;
}
```

- Para acomodar la configuración de las propiedades, los encabezados y el cuerpo de un mensaje que no se pueden encapsular genéricamente dentro del convertidor, la interfaz MessagePostProcessor brinda acceso al mensaje después de que se haya convertido pero antes de que se envíe.

```
public void sendWithConversion(Map map) {
   jmsTemplate.convertAndSend("testQueue", map, new MessagePostProcessor() {
      public Message postProcessMessage(Message message) throws JMSException {
         message.setIntProperty("AccountID", 1234);
         message.setJMSCorrelationID("123-00001");
         return message;
      }
   });
}
```

© JMA 2020. All rights reserved

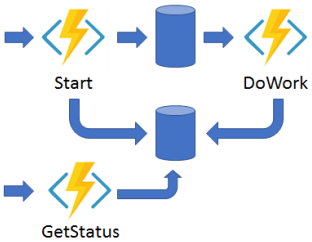
WebHooks

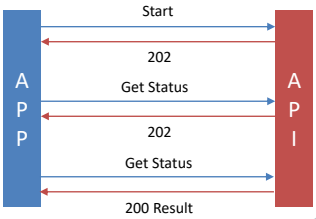
- Los webhooks son eventos HTTP que desencadenan acciones. Su nombre se debe a que funcionan como «enganches» de los programas en Internet y casi siempre se utilizan para la comunicación entre sistemas. Son la manera más sencilla de obtener un aviso cuando algo ocurre en otro sistema y para el intercambio de datos entre aplicaciones web, permitiendo las llamada asíncronas en HTTP.
- Un webhook es una retro llamada HTTP, una solicitud HTTP GET/POST insertada en una página web, que interviene cuando ocurre algo (una notificación de evento a través de HTTP GET/POST).
- Los webhooks se utilizan para las notificaciones en tiempo real (con los datos del evento como parámetros o en cuerpo en JSON o XML) a una determinada dirección http:// o https://, que puede:
  - almacenar los datos del evento en JSON o XML
  - generar una respuesta que permita actualizarse al sistema donde se produce el evento
  - ejecutar un proceso en el sistema receptor del evento (Ej: enviar un correo electrónico)
- Los webhook están pensados para su utilización desde páginas web y sus diferentes consumidores: navegadores, correo electrónico, webapps, …
- Un ejemplo típico es su utilización en correos electrónico de marketing para notificar al servidor que debe enviar un nuevo correo electrónico porque el usuario a abierto el mensaje.
- Pueden considerarse una versión especializada y simplificada de los servicios REST (solo GET/POST).

© JMA 2020. All rights reserved

WebHooks

- El patrón Async HTTP APIs soluciona el problema de coordinar el estado de las operaciones de larga duración con los clientes externos. Una forma habitual de implementar este patrón es que un punto de conexión HTTP desencadene la acción de larga duración. A continuación, el cliente se redirige a un punto de conexión de estado al que sondea para saber el estado actual del proceso y cuando finaliza la operación.
- Un endpoint HTTP desencadena el proceso con la acción de larga duración y devuelve inmediatamente un 202 si la petición es correcta y, opcionalmente, como carga útil las URLs de los endpoints de estado y control. Las peticiones incorrectas recibirán el 4XX apropiado.
- El cliente sondea periódicamente el endpoint de estado y obtiene:
  - 202 con el estado actual del proceso como carga útil mientras el proceso este en marcha.
  - 200 con el estado final (correcto o fallido) del proceso como carga útil cuando haya finalizado la operación.
- El endpoint HTTP desencadenador puede suministrar endpoints adicionales para pausar, reanudar, cancelar/terminar, reiniciar o enviar eventos al proceso en marcha.

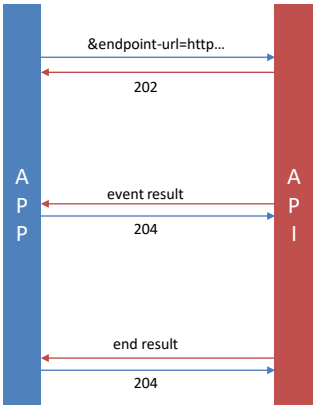




© JMA 2020. All rights reserved

WebHooks

- El patrón Reverse API soluciona el problema del sondeo periódico, las API inversas invierten esta situación, para que sea la API invocada la que notifique automáticamente cuando se ha producido un determinado evento. El cliente debe crear su propia API para recibir las notificaciones.
- Al realizar la petición al endpoint HTTP desencadenador, se suministra un endpoint propio para que el proceso desencadenado pueda notificar cuándo ocurre algo de interés. Le da la vuelta a la comunicación, el cliente pasa a ser servidor y el servidor a cliente. El desencadenador devuelve inmediatamente un 202 si la petición es correcta o el 4XX apropiado se es incorrecta.
- Este esquema de funcionamiento tiene muchas ventajas en ambos extremos:
  - **Ahorro de recursos y tiempo**: con el sondeo se harán muchas llamadas "para nada", que no devolverán información relevante. Los dos extremos gastan recursos para hacer y responder a muchas llamadas que no tienen utilidad alguna (no nos llame, ya les llamaremos).
  - **Eliminación de los retrasos**: la aplicación usuaria recibirá una llamada en el momento exacto en el que se produce y no tendrá retrasos al próximo sondeo.
  - **Velocidad de las llamadas**: generalmente la llamada que se hace a un webhook es muy rápida porque solo se envía una pequeña información sobre el evento y se suele procesar asíncronamente. Muchas veces ni siquiera se espera por el resultado: se hace una llamada del tipo "fire and forget" (o sea, dispara y olvídate), pues se trata de notificar el evento y listo.



© JMA 2020. All rights reserved

WEBSOCKET Y STOMP

© JMA 2020. All rights reserved

WebSocket

- El protocolo WebSocket, RFC 6455, proporciona un canal de comunicación bidireccional y full-dúplex, entre el cliente y el servidor, a través de una única conexión TCP. Es un protocolo TCP diferente de HTTP, pero está diseñado para funcionar sobre HTTP, utilizando los puertos 80 y 443 y permitiendo la reutilización de las reglas de firewall existentes.
- La API de WebSocket ha sido normalizada por el W3C, mientras que el protocolo WebSocket fue normalizado por la IETF como el RFC 6455.
- WebSockets es una tecnología avanzada que hace posible abrir una sesión de comunicación interactiva entre el navegador del usuario y un servidor. Con la API de navegador, se puede enviar mensajes a un servidor y recibir respuestas controladas por eventos sin tener que consultar al servidor para una respuesta: una vez establecida la conexión por parte del cliente, el servidor puede hacer envíos al cliente sin esperar a una petición.

© JMA 2020. All rights reserved

HTTP frente a WebSocket

- Aunque WebSocket está diseñado para ser compatible con HTTP y comienza con una solicitud HTTP, es importante comprender que los dos protocolos conducen a arquitecturas y modelos de programación de aplicaciones muy diferentes.
- En HTTP y REST, una aplicación se modela como muchas URL. Para interactuar con la aplicación, los clientes acceden a esas URL, estilo solicitud-respuesta. Los servidores enrutan las solicitudes al controlador apropiado en función de la URL, el método HTTP y los encabezados.
- Por el contrario, en WebSockets, generalmente solo hay una URL para la conexión inicial. Posteriormente, todos los mensajes de la aplicación fluyen en esa misma conexión TCP. Esto apunta a una arquitectura de mensajería asincrónica, impulsada por eventos, completamente diferente.
- WebSocket también es un protocolo de transporte de bajo nivel que, a diferencia de HTTP, no prescribe ninguna semántica para el contenido de los mensajes. Eso significa que no hay forma de enrutar o procesar un mensaje a menos que el cliente y el servidor acuerden la semántica del mensaje.
- Los clientes y servidores de WebSocket pueden negociar el uso de un protocolo de mensajería de nivel superior (por ejemplo, STOMP), a través del Sec-WebSocket-Protocolencabezado en la solicitud de protocolo de enlace HTTP. En ausencia de eso, necesitan llegar a sus propias convenciones.

© JMA 2020. All rights reserved

WebSocket

- El protocolo WebSocket define un prefijo ws:// y wss:// para indicar una conexión WebSocket y Websocket Secure, respectivamente. Una interacción de WebSocket comienza con una solicitud HTTP que usa el encabezado HTTP Upgrade para actualizar o cambiar al protocolo WebSocket:

```
GET /chat HTTP/1.1
Host: localhost:8080
Connection: Upgrade
Upgrade: websocket
Sec-WebSocket-Version: 13
Sec-WebSocket-Key: Uc9l9TMkWGbHFD2qnFHltg==
Origin: http://localhost:8080
```

- En lugar del código de estado 200 habitual, un servidor compatible con WebSocket devuelve:

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: upgrade
Sec-WebSocket-Accept: DoDFpVvCTJDVTPPfR9lEFDIt5A0=
Sec-WebSocket-Extensions: permessage-deflate;client
```

© JMA 2020. All rights reserved

Casos de uso

- WebSockets puede hacer que una página web sea dinámica e interactiva, es especialmente bueno para servicios que requieren intercambio de información continua, por ejemplo juegos en línea, sistemas de negocios en tiempo real, entre otros. Sin embargo, en muchos casos, una combinación de transmisión Ajax y HTTP o un sondeo largo puede proporcionar una solución simple y efectiva.
- La latencia por sí sola no es un factor decisivo. Si el volumen de mensajes es relativamente bajo (por ejemplo, al monitorizar fallos en la red), la transmisión o el sondeo HTTP pueden brindar una solución efectiva. Es la combinación de baja latencia, alta frecuencia y alto volumen lo que hace el mejor caso para el uso de WebSocket.
- Hay que tener en cuenta también que en Internet, los proxies restrictivos que están fuera de nuestro control pueden impedir las interacciones de WebSocket, ya sea porque no están configurados para transmitir el encabezado Upgrade o porque cierran conexiones de larga duración que parecen inactivas. Esto significa que el uso de WebSocket para aplicaciones internas dentro del firewall es una decisión más sencilla que para aplicaciones públicas.
- Al mantener conexiones abiertas son costosas en recursos del servidor (cuando hay muchos usuarios), no son fáciles de testear y hay hacerlas tolerante a fallos manualmente.

© JMA 2020. All rights reserved

WebSocket en Spring

- Spring Framework proporciona una API de WebSocket que se puede usar para escribir aplicaciones del lado del cliente y del servidor que manejen mensajes de WebSocket.
- Añadir al proyecto:
  - Messaging > WebSocket
- Crear un servidor WebSocket es tan simple como implementar WebSocketHandler, probablemente extendiendo TextWebSocketHandler o BinaryWebSocketHandler, y registrarlo en la configuración al habilitar los WebSockets.

```
@Configuration
@EnableWebSocket
public class WebSocketConfig implements WebSocketConfigurer {
    @Autowired
    SocketTextHandler cliente;
    public void registerWebSocketHandlers(WebSocketHandlerRegistry registry) {
        registry.addHandler(cliente, "/chat");
    }
}
```

© JMA 2020. All rights reserved

TextWebSocketHandler

```
@Component
public class SocketTextHandler extends TextWebSocketHandler {
    @Autowired SocketService clientes;
    @Override
    public void afterConnectionEstablished(WebSocketSession session) throws Exception {
        if (session.getUri().getQuery() == null) {
            session.close(new CloseStatus(1006, "Falta el nombre de usuario"));
        } else {
            clientes.add(session.getUri().getQuery(), session);
            session.sendMessage(new TextMessage("Servidor: Te has conectado como " + session.getUri().getQuery()));
        }
        super.afterConnectionEstablished(session);
    }
    @Override
    public void afterConnectionClosed(WebSocketSession session, CloseStatus status) throws Exception {
        clientes.remove(session);
        super.afterConnectionClosed(session, status);
    }
    @Override
    public void handleTextMessage(WebSocketSession session, TextMessage message)  throws InterruptedException, IOException {
        clientes.broadcast(clientes.getName(session) + ": " + message.getPayload());
    }
}
```

© JMA 2020. All rights reserved

Cliente JavaScript

```
<script>
let socket = null;
const connect = () => {
	socket = new WebSocket(`ws://${location.host}/chat`);
	socket.onopen = () => show("[open] Conexión establecida");
	socket.onmessage = event => show(`[message] ${event.data}`);
	socket.onclose = event => {
		show(`[close] Conexión cerrada ${event.wasClean ? 'limpiamente' : 'con problemas'}, código: ${event.code} ${event.reason}`);
	}
	socket.onerror = event => show(`[error]: ${JSON.stringify(error)}`);
}
const send = message => {
	if(socket && socket.readyState === 1)
		socket.send(message)
}
const disconnect = () => {
	if(socket && socket.readyState === 1)
		socket.close()
}
</script>
```

© JMA 2020. All rights reserved

Cliente Java

```
import org.springframework.web.socket.TextMessage;
import org.springframework.web.socket.WebSocketSession;
import org.springframework.web.socket.client.WebSocketClient;
import org.springframework.web.socket.client.standard.StandardWebSocketClient;
import org.springframework.web.socket.handler.TextWebSocketHandler;

	private WebSocketSession client;

		if (client == null || !client.isOpen()) {
			WebSocketClient ws = new StandardWebSocketClient();
			client = ws.execute(new TextWebSocketHandler(), WEBSOCKET_SERVER_URL).get();
		}
		if(client.isOpen()) {
			client.sendMessage(new TextMessage("[" + message.getName() + "]: " + message.getText()));
		} else {
			log.error("Sesion cerrada");
		}
```

© JMA 2020. All rights reserved

SockJS

- En la Internet pública, los proxies restrictivos fuera de nuestro control pueden impedir las interacciones de WebSocket, ya sea porque no están configurados para transmitir el encabezado Upgrade o porque cierran conexiones de larga duración que parecen estar inactivas.
- La solución a este problema es la emulación de WebSocket, es decir, intentar usar WebSocket primero y luego recurrir a técnicas basadas en HTTP que emulan una interacción de WebSocket y exponen la misma API de nivel de aplicación.
- En la pila de servlet, Spring Framework proporciona soporte tanto para el servidor (como para el cliente) para el protocolo SockJS.
- El objetivo de SockJS es permitir que las aplicaciones usen una API de WebSocket pero recurran a alternativas que no sean de WebSocket cuando sea necesario en tiempo de ejecución, sin necesidad de cambiar el código de la aplicación.
- SockJS consta de:
  - El protocolo SockJS definido entre el cliente en el navegador y sus contrapartes del lado del servidor.
  - El cliente JavaScript SockJS: una biblioteca cliente para usar en navegadores.
  - Implementaciones del servidor SockJS, incluida una en el módulo spring-websocket de Spring Framework.
  - Un cliente SockJS Java, también incluida en el módulo spring-websocket (desde la versión 4.1).

© JMA 2020. All rights reserved

SockJS

- El cliente SockJS comienza enviando GET /info para obtener información básica del servidor. Después de eso, debe decidir qué transporte utilizar. Si es posible, se utiliza WebSocket. Si no, en la mayoría de los navegadores, hay al menos una opción de transmisión HTTP (HTTP Streaming, Server Sent Events). De lo contrario, se utiliza el sondeo HTTP largo (HTTP Long Polling).
- Se puede activar al registrar el WebSocketHandler:

```
registry.addHandler(cliente, "/chat").setAllowedOrigins("*");
```

- Para acceder desde el navegador:

```
<script src="https://cdn.jsdelivr.net/npm/sockjs-client@1/dist/sockjs.min.js"></script>
<script>
var socket = new SockJS('/chat');
socket.onopen = () => …
socket.onmessage = event => …
socket.onclose = event => …
socket.onerror = event => …
socket.send(message)
socket.close()
```

© JMA 2020. All rights reserved

Orígenes permitidos

- A partir de Spring Framework 4.1.5, el comportamiento predeterminado de WebSocket y SockJS es aceptar solo solicitudes del mismo origen cuando el clientes es un navegador. También es posible permitir todos o una lista específica de orígenes. Nada impide que otros tipos de clientes modifiquen el valor del encabezado Origin.
- Los tres comportamientos posibles son:
  - Permitir solo solicitudes del mismo origen (predeterminado): en este modo, cuando SockJS está habilitado, el encabezado de respuesta HTTP de Iframe X-Frame-Options se establece en SAMEORIGIN y el transporte JSONP está deshabilitado, ya que no permite verificar el origen de una solicitud.
  - Permitir una lista específica de orígenes: cada origen permitido debe comenzar con http:// o https://. En este modo, cuando SockJS está habilitado, el transporte de IFrame está deshabilitado.
  - Permitir todos los orígenes: para habilitar este modo, se debe proporcionar el * como valor de origen permitido. En este modo, todos los transportes están disponibles.
- Se puede configurar los orígenes permitidos al registrar el WebSocketHandler:

```
registry.addHandler(cliente, "/chat").setAllowedOrigins("*");
```

© JMA 2020. All rights reserved

STOMP sobre WebSocket

- El protocolo WebSocket define dos tipos de mensajes (texto y binario), pero su contenido no está definido. El protocolo define un mecanismo para que el cliente y el servidor negocien un subprotocolo (es decir, un protocolo de mensajería de nivel superior) para usar sobre WebSocket para definir qué tipo de mensajes puede enviar cada uno, cuál es el formato, el contenido de cada mensaje, y así sucesivamente. El uso de un subprotocolo es opcional pero, de cualquier manera, el cliente y el servidor deben acordar algún protocolo que defina el contenido del mensaje.
- STOMP (Protocolo de mensajería orientado a texto simple) define un formato de conexión interoperable para que cualquiera de los clientes STOMP disponibles pueda comunicarse con cualquier intermediario de mensajes STOMP para proporcionar una interoperabilidad de mensajería fácil y generalizada entre lenguajes y plataformas. Se creó originalmente para que lenguajes de secuencias de comandos (como Ruby, Python y Perl) se conectasen a intermediarios de mensajes empresariales. Está diseñado para abordar un subconjunto mínimo de patrones de mensajería de uso común y se puede usar sobre cualquier protocolo de red de transmisión bidireccional confiable, como TCP y WebSocket. Aunque es un protocolo orientado a texto, las cargas de mensajes pueden ser de texto o binarias.

© JMA 2020. All rights reserved

Protocolo orientado a texto

- Los clientes pueden usar los comandos SEND o SUBSCRIBE para enviar o suscribirse a mensajes, junto con un encabezado destination que describe de qué se trata el mensaje y quién debe recibirlo. Esto habilita un mecanismo simple de publicación y suscripción que puede usar para enviar mensajes a través del intermediario a otros clientes conectados o para enviar mensajes al servidor para solicitar que se realice algún trabajo.

```
CONNECT
login:guest
passcode:guest

    CONNECTED
    session:1092296064

SEND
destination:/queue/test

Hello from TCP!

SUBSCRIBE
id:sub-1
destination:/topic/price.stock.*
```

© JMA 2020. All rights reserved

STOMP con Spring

- Cuando utiliza la compatibilidad con STOMP de Spring, la aplicación Spring WebSocket actúa como intermediario de STOMP para los clientes. Los mensajes se enrutan a los métodos de manejo de mensajes de un @Controller o a un intermediario en memoria simple que realiza un seguimiento de las suscripciones y transmite mensajes a los usuarios suscritos.
- También puede configurar Spring para trabajar con un agente STOMP dedicado (como RabbitMQ, ActiveMQ y otros) para la transmisión real de mensajes. En ese caso, Spring mantiene conexiones TCP con el intermediario, le transmite mensajes y pasa mensajes desde él a los clientes WebSocket conectados. Por lo tanto, las aplicaciones web de Spring pueden confiar en la seguridad unificada basada en HTTP, la validación común y un modelo de programación familiar para el manejo de mensajes.
- El uso de STOMP como subprotocolo permite que Spring Framework y Spring Security proporcionen un modelo de programación más rico en comparación con el uso de WebSockets sin procesar. La siguiente es una lista de beneficios:
  - No es necesario inventar un protocolo de mensajería personalizado y un formato de mensaje.
  - Los clientes STOMP, incluido un cliente Java en Spring Framework, ya están disponibles.
  - Se pueden usar intermediarios de mensajes (como RabbitMQ o ActiveMQ y otros) para administrar suscripciones y difundir mensajes.
  - La lógica de la aplicación se puede organizar en cualquier cantidad de instancias @Controller y los mensajes se pueden enrutar a ellas en función del encabezado de destino de STOMP en lugar de manejar los mensajes WebSocket sin procesar con un único WebSocketHandler para una conexión determinada.
  - Se puede usar Spring Security para proteger los mensajes en función de los destinos y tipos de mensajes de STOMP.

© JMA 2020. All rights reserved

Configurar el servidor

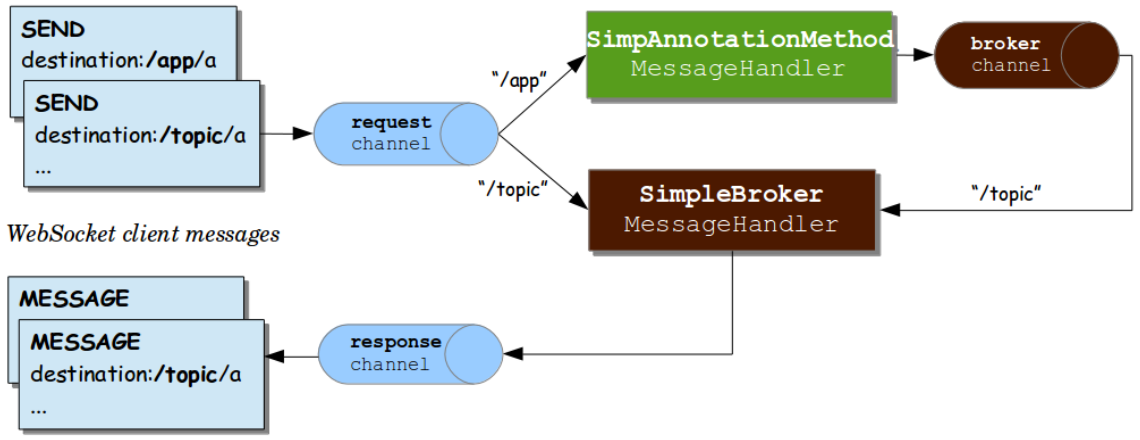
- La compatibilidad con STOMP sobre WebSocket está disponible en los módulos spring-messaging y spring-websocket.
- Para configurar el servidor WebSocket subyacente:

```
@Configuration
@EnableWebSocketMessageBroker
public class StompWebSocketConfig implements WebSocketMessageBrokerConfigurer {
  @Override public void registerStompEndpoints(StompEndpointRegistry registry) {
    registry.addEndpoint("/chat").withSockJS();
  }
  @Override public void configureMessageBroker(MessageBrokerRegistry config) {
    config.setApplicationDestinationPrefixes("/app");
    config.enableSimpleBroker("/topic", "/queue");
  }
}
```

- Donde /chat es la URL HTTP para el punto final al que un cliente WebSocket (o SockJS) necesita conectarse para el protocolo de enlace WebSocket. Los mensajes STOMP cuyo encabezado de destino comienza con /app se enrutan a los métodos @MessageMapping de los @Controller. Se habilita el agente de mensajes integrado para suscripciones y transmisiones, para que enrute los mensajes cuyo encabezado de destino comience con /topic o /queue a los métodos @SendTo de los @Controller.

© JMA 2020. All rights reserved

Flujo de mensajes



© JMA 2020. All rights reserved

Controladores anotados

- Las aplicaciones pueden usar clases anotadas con @Controller para manejar mensajes de clientes. Dichas clases pueden declarar métodos:
  - @MessageMapping: para anotar métodos que enrutan mensajes en función de su destino.
  - @SubscribeMapping: limita la asignación a los mensajes de suscripción únicamente.
  - @MessageExceptionHandler: para manejar las excepciones de los métodos @MessageMapping.

```
@Controller
@RequiredArgsConstructor
public class ChatController {
   private final StompService socketService;
   @MessageMapping("/chat")
   @SendTo("/topic/messages")
   public MessageDto sendMessage(MessageDto message, @Header("simpSessionId") String sessionId) {
      socketService.saveSession(sessionId, message.getName());
      return message;
   }
}
```

© JMA 2020. All rights reserved

SimpMessagingTemplate

- Cualquier componente de la aplicación puede enviar mensajes al brokerChannel. La forma más sencilla de hacerlo es inyectar un SimpMessagingTemplate y usarlo para enviar mensajes.

```
@Controller
public class GreetingController {
   @Autowired
   private SimpMessagingTemplate template;
   @PostMapping(path="/saluda")
   public void send(String name) {
      this.template.convertAndSend("/topic/messages", "Hola " + name);
   }
}
```

© JMA 2020. All rights reserved

Simple Broker

- El intermediario de mensajes simple (Simple Broker) incorporado maneja las solicitudes de suscripción de los clientes, las almacena en la memoria y transmite mensajes a los clientes conectados que tienen destinos coincidentes. El bróker admite destinos similares a las rutas.

```
@Configuration
@EnableWebSocketMessageBroker
public class StompWebSocketConfig implements WebSocketMessageBrokerConfigurer {
   @Lazy @Autowired
   private TaskScheduler messageBrokerTaskScheduler;
   @Override
   public void configureMessageBroker(MessageBrokerRegistry registry) {
      registry.enableSimpleBroker("/queue/", "/topic/")
            .setHeartbeatValue(new long[] {10000, 20000})
            .setTaskScheduler(this.messageBrokerTaskScheduler);
      :
   }
}
```

© JMA 2020. All rights reserved

External Broker

- El intermediario simple es excelente para comenzar, pero admite solo un subconjunto de comandos STOMP (no admite confirmaciones, recibidos y algunas otras funciones), se basa en un ciclo simple de envío de mensajes y no es adecuado para la agrupación.
- Como alternativa, se pueden actualizar las aplicaciones para utilizar un intermediario de mensajes externo (como RabbitMQ, ActiveMQ y otros) con todas las funciones. El intermediario debe ejecútarse con la compatibilidad con STOMP habilitada.
- Para habilitar el reenvío al intermediario de mensajes externo en la configuración:

```
@Configuration
@EnableWebSocketMessageBroker
public class StompWebSocketConfig implements WebSocketMessageBrokerConfigurer {
   :
   @Override public void configureMessageBroker(MessageBrokerRegistry config) {
      config.setApplicationDestinationPrefixes("/app");
      config.enableStompBrokerRelay("/topic", "/queue");
   }
}
```

© JMA 2020. All rights reserved

Cliente JavaScript

```
<script>
    let stompClient = null;
    let unsubscribe = null;
    const connect = () => {
        stompClient = Stomp.client(`ws://${location.host}/chat`);
        unsubscribe = stompClient.connect({}, frame => {
            stompClient.subscribe('/topic/messages', function (response) {
                let messaje = JSON.parse(response.body)
                show(`${messaje.name}: ${messaje.text}`)
            });
        }, error => show(`[error]: ${JSON.stringify(error)}`));
    }
    const send = message => {
        stompClient.send("/example/chat", {}, JSON.stringify({ 'name': username, text: input.value }));
    }
    const disconnect = () => {
        if (unsubscribe) unsubscribe();
        stompClient.disconnect(() => show(`[close] Disconnect`));
    }
</script>
```

© JMA 2020. All rights reserved

WebJars

- Dependencias (en Maven):

```
<dependency>
    <groupId>org.webjars</groupId>
    <artifactId>webjars-locator-core</artifactId>
</dependency>
<dependency>
    <groupId>org.webjars</groupId>
    <artifactId>sockjs-client</artifactId>
    <version>1.0.2</version>
</dependency>
<dependency>
    <groupId>org.webjars</groupId>
    <artifactId>stomp-websocket</artifactId>
    <version>2.3.4</version>
</dependency>
```

- Referenciado (en HTML):

```
<script src="/webjars/sockjs-client/sockjs.min.js"></script>
<script src="/webjars/stomp-websocket/stomp.min.js"></script>
```

© JMA 2020. All rights reserved

Cliente Java

```
WebSocketClient client = new StandardWebSocketClient();
stompClient = new WebSocketStompClient(client);
stompClient.setMessageConverter(new MappingJackson2MessageConverter());
stompClient.connectAsync(STOMP_SERVER_URL, new StompSessionHandlerAdapter() {
   private static final Logger log = LoggerFactory.getLogger("StompSessionHandlerAdapter");
   @Override
   public void afterConnected(StompSession session, StompHeaders connectedHeaders) {
      session.subscribe("/topic/messages", this);
      session.send("/example/chat", new MessageDto("[REMOTE]", "Me he conectado!!!"));
   }
   @Override
   public void handleException(StompSession session, StompCommand command, StompHeaders headers, byte[] payload, Throwable exception) {
      log.error("Got an exception", exception);
   }
   @Override public Type getPayloadType(StompHeaders headers) { return MessageDto.class; }
   @Override
   public void handleFrame(StompHeaders headers, Object payload) {
      MessageDto msg = (MessageDto) payload;
      try {
         clientes.broadcast("[STOMP " + msg.getName() + "]: " + msg.getText());
      } catch (IOException e) { e.printStackTrace(); }
   }
});
```

© JMA 2020. All rights reserved

CONSISTENCIA DE DATOS

© JMA 2020. All rights reserved

Transacciones

- Una transacción es una secuencia de operaciones realizadas como una sola unidad lógica de trabajo.
- Una unidad lógica de trabajo debe exhibir cuatro propiedades, conocidas como propiedades ACID (atomicidad, coherencia, aislamiento y durabilidad), para ser calificada como transacción:
  - Atomicidad: Una transacción debe ser una unidad atómica de trabajo, tanto si se realizan todas sus modificaciones en los datos, como si no se realiza ninguna de ellas.
  - Coherencia**:** Cuando finaliza, una transacción debe dejar todos los datos en un estado coherente, es decir, se deben cumplir todas las reglas de integridad de todos los datos.
  - aIslamiento**:** Las modificaciones realizadas por transacciones simultáneas se deben aislar de las modificaciones llevadas a cabo por otras transacciones simultáneas.
  - Durabilidad**:** Una vez concluida una transacción, sus efectos son permanentes en el sistema. Las modificaciones persisten aún en el caso de producirse un error del sistema.

© JMA 2020. All rights reserved

Transacciones distribuidas

- Una transacción distribuida es una transacción que afecta a varios recursos. Para que una transacción distribuida se confirme, todos los participantes deben garantizar que los cambios en los datos serán permanentes. Los cambios deben conservarse a pesar de bloqueos del sistema u otros eventos imprevistos. Si alguno de los participantes no cumple esta garantía, toda la transacción da error y se revertirán los cambios en los datos en el ámbito de la transacción.
- El protocolo 2PC se denomina así porque emplea las dos fases siguientes antes de confirmar la operación realizada:
  - Fase 1: Preparación. En esta fase, cada uno de los programas implicados en la transacción envía un mensaje al administrador de la TD, como el Coordinador de transacciones distribuidas (MS DTC), para informar que está listo para realizar su parte de la operación y de que es capaz de hacerlo o no puede. Esta fase se conoce como preparación, porque los programas están preparados para confirmar los cambios o revertirlos. Si el administrador de la TD recibe la confirmación de cada uno de los programas implicados, pasa a la fase 2.
  - Fase 2: confirmación o reversión. En esta fase, el administrador de la TD indica a cada uno de los programas que confirme o revierta todos los cambios solicitados como parte de la transacción. Una reversión ejecutada correctamente debe devolver el sistema a su estado original.

© JMA 2020. All rights reserved

Patrón: Base de datos compartida

- Motivación:
  - Ha aplicado el patrón de arquitectura de Microservicios. La mayoría de los servicios necesitan conservar los datos en algún tipo de base de datos..
- Intención:
  - ¿Cuál es la arquitectura de la base de datos en una aplicación de microservicios?
- Requisitos:
  - Los servicios se deben acoplar libremente para que se puedan desarrollar, implementar y escalar de forma independiente
  - Algunas transacciones de negocio deben imponer invariantes que abarcan múltiples servicios, otras deben actualizar los datos propiedad de múltiples servicios.
  - Algunas transacciones de negocio necesitan consultar datos de múltiples servicios.
  - Algunas consultas deben unir datos que son propiedad de múltiples servicios.
  - Las bases de datos a veces se deben replicar y fragmentar para escalar.
  - Los diferentes servicios tienen diferentes requisitos de almacenamiento de datos. Para algunos servicios, una base de datos relacional es la mejor opción, otros pueden necesitar una base de datos NoSQL como MongoDB o Redis.
- Solución:
  - Utilizar una base de datos (única) que sea compartida por múltiples servicios. Cada servicio accede libremente a los datos que son propiedad de otros servicios que utilizan transacciones ACID locales.

© JMA 2020. All rights reserved

Patrón: Base de datos compartida

- Implementación:
  - Crear servicios con Spring Boot y Spring Data
- Consecuencias:
  - Los beneficios de este patrón son:
    - Un desarrollador utiliza transacciones ACID familiares y directas para imponer la consistencia de los datos
    - Una sola base de datos es más simple de operar
  - Los inconvenientes de este patrón son:
    - Acoplamiento en tiempo de desarrollo: un desarrollador que trabaja un servicio necesitará coordinar los cambios de esquema con los desarrolladores de otros servicios que acceden a las mismas tablas. Dichos cambios implican un despliegue coordinado de todos los servicios afectados. Este acoplamiento y la coordinación adicional ralentizarán el desarrollo.
    - Acoplamiento en tiempo de ejecución: dado que todos los servicios acceden a la misma base de datos, pueden interferir entre sí y provocar fallos en cascada.
    - La base de datos única puede no satisfacer los requisitos de almacenamiento y acceso a datos de todos los servicios.
- Patrones relacionados:
  - El patrón de Base de datos por servicio es un patrón alternativo.

© JMA 2020. All rights reserved

Patrón: Base de datos por servicio

- Motivación:
  - Ha aplicado el patrón de arquitectura de Microservicios. La mayoría de los servicios necesitan conservar los datos en algún tipo de base de datos.
- Intención:
  - ¿Cuál es la arquitectura de la base de datos en una aplicación de microservicios?
- Requisitos:
  - Los servicios se deben acoplar libremente para que se puedan desarrollar, implementar y escalar de forma independiente
  - Algunas transacciones de negocio deben imponer invariantes que abarcan múltiples servicios, otras deben actualizar los datos propiedad de múltiples servicios.
  - Algunas transacciones de negocio necesitan consultar datos que son propiedad de múltiples servicios.
  - Algunas consultas deben unir datos que son propiedad de múltiples servicios.
  - Las bases de datos a veces se deben replicar y fragmentar para escalar.
  - Los diferentes servicios tienen diferentes requisitos de almacenamiento de datos. Para algunos servicios, una base de datos relacional es la mejor opción. Otros servicios pueden necesitar una base de datos NoSQL como MongoDB, que es buena para almacenar datos complejos y no estructurados, o Redis, que está diseñada para almacenar y consultar datos de en memoria.

© JMA 2020. All rights reserved

Patrón: Base de datos por servicio

- Solución:
  - Mantener los datos persistentes de cada microservicio en privados para el servicio y accesibles solo a través de su API. Las transacciones de un servicio solo involucran su base de datos.
  - La base de datos del servicio es parte de la implementación de ese servicio y no se puede acceder directamente por otros servicios.
  - Hay algunas formas diferentes de mantener la privacidad de los datos persistentes de un servicio. No es necesario aprovisionar un servidor de base de datos para cada servicio. Por ejemplo, si está utilizando una base de datos relacional, las opciones son:
    - Tablas privadas por servicio: cada servicio posee un conjunto de tablas a las que solo debe acceder ese servicio
    - Esquema por servicio: cada servicio tiene un esquema de base de datos que es privado para ese servicio
    - Servidor de base de datos por servicio: cada servicio tiene su propio servidor de base de datos.
  - Las tablas privadas por servicio y el esquema por servicio tienen la sobrecarga más baja. Usar un esquema por servicio es atractivo ya que hace que la propiedad sea más clara. Algunos servicios de alto rendimiento pueden necesitar su propio servidor de base de datos.
  - Es una buena idea crear barreras que impongan esta modularidad. Se debería asignar un usuario de base de datos diferente a cada servicio y utilizar control de acceso y permisos de la base de datos, dado que los desarrolladores de otros servicios siempre estarán tentados a pasar por alto la API de un servicio y acceder a sus datos directamente.
- Implementación:
  - Crear servicios con Spring Boot y Spring Data

© JMA 2020. All rights reserved

Patrón: Base de datos por servicio

- Consecuencias:
  - El uso de una base de datos por servicio tiene los siguientes beneficios:
    - Ayuda a asegurar que los servicios estén acoplados libremente. Los cambios en la base de datos de un servicio no afectan a ningún otro servicio.
    - Cada servicio puede utilizar el tipo de base de datos que mejor se adapte a sus necesidades
  - El uso de una base de datos por servicio tiene los siguientes inconvenientes:
    - Implementar transacciones distribuidas que abarcan múltiples servicios no es sencillo. Es mejor evitar las transacciones distribuidas debido al teorema de CAP: es imposible en un sistema distribuido garantizar simultáneamente la consistencia (Consistency), la disponibilidad (Availability) y tolerancia al particionado (Partition Tolerance). Además, muchas bases de datos modernas (NoSQL) no las admiten. La mejor solución es usar el patrón Saga: Los servicios publican eventos cuando actualizan datos y otros servicios se suscriben a eventos y actualizan sus datos en respuesta.
    - Implementar consultas que unen datos que ahora están en múltiples bases de datos es un desafío.
    - Complejidad de gestionar múltiples bases de datos SQL y NoSQL
- Patrones relacionados:
  - El patrón de arquitectura de microservicio crea la necesidad de este patrón
  - El patrón Saga es una forma útil de implementar transacciones eventualmente consistentes
  - El patrón de API de composición de la API y el de Segregación de responsabilidad de consultas y comandos (CQRS) son formas útiles de implementar consultas
  - El patrón de Base de datos compartida es un patrón alternativo.

© JMA 2020. All rights reserved

Patrón: Eventos de dominio

- Motivación:
  - Un servicio a menudo necesita publicar eventos cuando actualiza sus datos. Estos eventos pueden ser necesarios, por ejemplo, para actualizar una vista CQRS. Alternativamente, el servicio podría participar en una Saga basada en coreografía, que utiliza eventos para la coordinación.
- Intención:
  - ¿Cómo publica un servicio un evento cuando actualiza sus datos?
- Solución:
  - Organizar la lógica de negocios de un servicio como una colección de agregados DDD que emiten eventos de dominio cuando se crean o actualizan. El servicio publica estos eventos de dominio para que puedan ser consumidos por otros servicios.
- Patrones relacionados:
  - Los patrones Saga y CQRS crean la necesidad de este patrón.
  - El patrón Agregado se utiliza para estructurar la lógica empresarial.
  - El patrón de Bandeja de salida transaccional se utiliza para publicar eventos como parte de una transacción de base de datos
  - El aprovisionamiento de eventos se utiliza a veces para publicar eventos de dominio.

© JMA 2020. All rights reserved

Patrón: Saga

- Motivación:
  - Se ha aplicado el patrón base de datos por servicio. Cada servicio tiene su propia base de datos. Sin embargo, algunas transacciones abarcan varios servicios, por lo que necesita un mecanismo para garantizar la consistencia de los datos en todos los servicios. Dado que datos se encuentran en diferentes bases de datos, la aplicación no puede simplemente usar una transacción local ACID.
- Intención:
  - ¿Cómo mantener la consistencia de los datos en todos los servicios?
- Requisitos:
  - Las transacciones de Confirmación en dos fases (2PC) no son una opción
- Solución:
  - Implementar cada transacción que abarque múltiples servicios como una saga (de películas o libros). Una saga es una secuencia de transacciones locales. Cada transacción local actualiza su base de datos local y publica un mensaje o evento para desencadenar la siguiente transacción local en la saga. Si una transacción local falla porque viola una regla de negocio, entonces la saga ejecuta una serie de transacciones compensatorias (reversión) que deshacen los cambios realizados por las transacciones locales anteriores.
  - Hay dos formas de coordinar las sagas:
    - Coreografía: cada transacción local publica eventos de dominio que activan transacciones locales en otros servicios
    - Orquestación: un orquestador (objeto) le dice a los participantes qué transacciones locales deben ejecutar

© JMA 2020. All rights reserved

Patrón: Saga

- Implementación:
  - Crear servicios con Spring Boot y Spring Data, Spring for RabbitMQ, …
- Consecuencias:
  - Este patrón tiene los siguientes beneficios:
    - Permite que una aplicación mantenga la consistencia de los datos en múltiples servicios sin usar transacciones distribuidas
  - Esta solución tiene los siguientes inconvenientes:
    - El modelo de programación es más complejo, se debe diseñar transacciones de compensación que deshagan explícitamente los cambios realizados anteriormente en una saga.
    - No permite utilizar bloqueos que eviten efectos secundarios a consecuencia de las transacciones de compensación.
  - También hay que abordar los siguientes temas:
    - Para ser confiable, un servicio debe actualizar su base de datos y publicar un mensaje/evento. No puede usar el mecanismo tradicional de una transacción distribuida que abarca la base de datos y el intermediario de mensajes. En su lugar, debe utilizar patrones adicionales.
- Patrones relacionados:
  - El patrón de base de datos por servicio crea la necesidad de este patrón.
  - Los patrones Abastecimiento de eventos y Eventos de aplicación son formas de actualizar de forma atómica el estado y publicar mensajes/eventos:
  - Una saga basada en coreografía puede publicar eventos usando agregados y eventos de dominio.

© JMA 2020. All rights reserved

Patrón: Event sourcing

- Motivación:
  - Un servicio normalmente necesita actualizar de forma atómica la base de datos y publicar mensajes / eventos. Por ejemplo cuando utiliza el patrón Saga. Para ser confiable, cada paso de una saga debe actualizar atómicamente la base de datos y publicar mensajes / eventos. Alternativamente, podría usar el patrón de evento de dominio, tal vez para implementar CQRS. En cualquier caso, no es viable utilizar una transacción distribuida que abarque la base de datos y el intermediario de mensajes para actualizar de forma atómica la base de datos y publicar mensajes / eventos.
- Intención:
  - ¿Cómo actualizar de forma fiable/atómica la base de datos y publicar mensajes/eventos?
- Requisitos:
  - Las transacciones de Confirmación en dos fases (2PC) no son una opción
- Solución:
  - Una buena solución es registrar el historial de la secuencia de eventos de cambio de una entidad de dominio desde el momento de su creación hasta el momento actual y hacer persistente dicha secuencia. Cuando el estado de una entidad de dominio cambia, se agrega un nuevo evento a la lista de eventos. Dado que guardar un evento es una sola operación, es inherentemente atómico. La aplicación reconstruye el estado actual de una entidad mediante la reproducción de los eventos.
  - Las aplicaciones persisten eventos en un almacén de eventos, que es una base de datos de eventos. El almacén de eventos tiene una API para agregar y recuperar eventos de una entidad. El almacén de eventos también se comporta como un intermediario de mensajes. Proporciona una API que permite a los servicios suscribirse a eventos. Cuando un servicio guarda un evento en el almacén de eventos, se entrega a todos los suscriptores interesados. Algunas entidades pueden tener una gran cantidad de eventos. Para optimizar la carga, una aplicación puede guardar periódicamente una instantánea del estado actual de una entidad. Para reconstruir el estado actual, se busca la instantánea más reciente y los eventos que han ocurrido desde esa instantánea. Como resultado, hay menos eventos para reproducir.

© JMA 2020. All rights reserved

Patrón: Event sourcing

- Implementación:
  - Crear servicios con Spring Boot y Spring Data, Spring for Kafka, …
  - Usar motores especializados: EventStoreDB, IBM Db2 Event Store, …
- Consecuencias:
  - El patrón tiene varios beneficios:
    - Resuelve uno de los problemas clave en la implementación de una arquitectura dirigida por eventos y permite publicar eventos de manera confiable cada vez que cambia el estado.
    - Debido a que persisten los eventos en lugar de los objetos de dominio, principalmente evita el problema de desajuste de impedancia relacional del objeto.
    - Proporciona un registro de auditoría 100% confiable de los cambios realizados en una entidad de dominio
    - Permite implementar consultas temporales que determinan el estado de una entidad en cualquier momento.
    - La lógica de negocios basada en la obtención de eventos consiste en entidades dominio débilmente acopladas que intercambian eventos. Esto hace que sea mucho más fácil migrar desde una aplicación monolítica a una arquitectura de microservicio.
  - El patrón también tiene varios inconvenientes:
    - Es un estilo de programación diferente y desconocido, por lo que hay una curva de aprendizaje.
    - El almacén de eventos es difícil de consultar ya que requiere consultas típicas para reconstruir el estado de las entidades dominio. Es probable que sea complejo e ineficiente. Como resultado, la aplicación debe utilizar Segregación de responsabilidad de consultas y comandos (CQRS) para implementar consultas. Esto, a su vez, significa que las aplicaciones deben manejar datos finalmente consistentes.
- Patrones relacionados:
  - Los patrones de Saga y Eventos Dominio crean la necesidad de este patrón.
  - El CQRS se debe utilizar a menudo con event sourcing.
  - El event sourcing implementa el patrón de registro de auditoría .

© JMA 2020. All rights reserved

Patrón: Segregación de responsabilidad de consultas y comandos (CQRS)

- Motivación:
  - Se han aplicado los patrones de arquitectura de microservicios y de base de datos por servicio. Como resultado, ya no es sencillo implementar consultas que unen datos de múltiples servicios. Si además se ha aplicado el patrón Event sourcing, los datos ya no se consultan fácilmente.
- Intención:
  - ¿Cómo implementar una consulta que recupera datos de múltiples servicios en una arquitectura de microservicio?
- Requisitos:
- Solución:
  - La idea básica es que puede dividir las operaciones de un sistema en dos categorías claramente separadas:
    - Consultas: Devuelven un resultado sin cambiar el estado del sistema y no tienen efectos secundarios.
    - Comandos: Cambian el estado de un sistema.
  - La solución pasa por dividir en dos subsistemas diferenciados, uno responsable de los comandos y otro responsable de las consultas. El sistema tradicional quedaría como el subsistema de comandos para el mantenimiento de los datos y se derivaría todas las consultas al subsistema de consultas. El subsistema de consultas contaría con su propio modelo de información y mecanismo de persistencia desnormalizado y optimizado para el tipo de consultas que tenga que atender, obtendría sus datos mediante mecanismos de sincronización.

© JMA 2020. All rights reserved

Patrón: Segregación de responsabilidad de consultas y comandos (CQRS)

- Implementación:
  - Crear servicios con Spring Boot y Spring Data, Spring for RabbitMQ, …
- Consecuencias:
  - Este patrón tiene los siguientes beneficios:
    - Soporta múltiples vistas desnormalizadas que son escalables y de alto rendimiento.
    - Separación de preocupaciones mejorada = modelos de comando y consulta más simples
    - Ambos subsistemas pueden evolucionar por separado y escalar; el mantenimiento y despliegue puede estar diferenciado.
  - Este patrón tiene los siguientes inconvenientes:
    - Mayor complejidad
    - Duplicación de datos con posibles inconsistencias
    - Replicación con retrasos / eventos inconsistentes
- Patrones relacionados:
  - El patrón de base de datos por servicio crea la necesidad de este patrón.
  - El patrón de API de composición es una solución alternativa.
  - El patrón de eventos del dominio generará los eventos.
  - CQRS se utiliza a menudo en combinación con el patrón Event sourcing

© JMA 2020. All rights reserved

Patrón: API de composición

- Motivación:
  - Se han aplicado los patrones de arquitectura de microservicios y de base de datos por servicio. Como resultado, ya no es sencillo implementar consultas que unen datos de múltiples servicios.
- Intención:
  - ¿Cómo implementar una consulta que recupera datos de múltiples servicios en una arquitectura de microservicio?
- Requisitos:
- Solución:
  - Crear un servicio REST que implemente las consultas como la composición de la invocación de los servicios que poseen los datos y la unión en memoria de los resultados.
- Implementación:
  - Crear servicios con Spring Boot y Spring Data, Spring for RabbitMQ, …
- Consecuencias:
  - Este patrón tiene los siguientes beneficios:
    - Es una forma sencilla de consultar datos en una arquitectura de microservicio.
  - Este patrón tiene los siguientes inconvenientes:
    - Algunas consultas darían resultado ineficaces, combinaciones en memoria de grandes conjuntos de datos.
- Patrones relacionados:
  - El patrón de base de datos por servicio crea la necesidad de este patrón.
  - El patrón CQRS de composición es una solución alternativa.

© JMA 2020. All rights reserved